

Contents

1	Implementation Plan — SCEC TPV26 (elastic) & TPV27 (Drucker–Prager viscoplastic) via <code>seas_spatial_dyn_driver</code>	1
1.1	0. Executive summary	2
1.2	1. Benchmark specification (condensed, verified)	5
1.3	2. Codebase architecture relevant to TPV26/27	8
1.4	3. Gap analysis	9
1.5	4. Detailed design	10
1.6	5. Phased implementation plan	19
1.7	6. Testing & verification strategy	30
1.8	7. Risks & open decisions	31
1.9	8. File manifest	31
1.10	Appendix A — Provenance	34

1 Implementation Plan – SCEC TPV26 (elastic) & TPV27 (Drucker–Prager viscoplastic) via `seas_spatial_dyn_driver`

- **Date:** 2026-06-28
- **Author:** Claude (code-explore + code-plan)
- **Worktree:** `.claude/worktrees/tpv26-27-design` (branch `worktree-tpv26-27-design`, based on HEAD `4e98bd0`)
- **Spec source:** `tpv26/benchmark_document/TPV26_27_Description_v13.pdf` (Barall, v13, 2014-01-09, 33 pp)
- **Status:** PLAN ONLY — no code changes implemented yet.
- **Revisions:** rev 1 (2026-06-28) initial plan. rev 2 (2026-06-28) §1.5/§4.4/§6/§7 revised against a direct read of the SeisSol off-fault-plasticity source (effective-stress storage, exact `computeRelaxTime` / `yieldFactor` / plastic-strain forms, per-step post-flux cadence, modal↔nodal nuance). rev 3 (2026-06-28) per user direction: (a) the **SCEC benchmark document is authoritative**, SeisSol is a cross-check only;
(b) nonlinear rheology refactored into a **separate, extensible BulkRheology constitutive path** (§4.4) — linear-elastic no-op default, Drucker–Prager first member, continuum damage–breakage (CDBM) reserved seat aligned with the companion CDBM port plan (`cdbm-damage-breakage-plan`). rev 4 (2026-07-08) **§5 rewritten into detailed per-phase contracts** (code structure + concrete unit tests per phase), re-derived against the current worktree by a 4-agent code-exploration pass; three drifted/superseded anchors corrected and flagged **[CORR]** in §5 (nucleation resolution is now free-function-based, `NucleationKind` has only 3 members, and the `#if 0` forced-rupture tests must be *adapted* not revived). Verified-current line anchors folded in. rev 5 (2026-07-08) per user direction — **off-fault plasticity is wanted in the quasi-dynamic (interseismic) path too, later**, not only coseismic. The DP constitutive math is therefore factored into an **embedding-agnostic shared core** (`constitutive/drucker_prager_core.{hpp,cpp}`) that the dynamic BulkRheology adapter wraps now and a future QD ConstitutiveModel

adapter reuses; per-node inelastic state is **cycle-persistent** and σ^0 is **injected** (new §4.4.8 seam). Dynamic-adapter only is built here; the QD adapter is deferred but rewrite-free.

One-line framing. TPV26 and TPV27 are *the same problem* (planar vertical surface-breaking strike-slip fault, depth-dependent initial stress, linear slip-weakening + forced-rupture nucleation, depth-dependent frictional cohesion) differing **only** in the off-fault bulk rheology: TPV26 = linear elastic, TPV27 = non-associative Drucker–Prager viscoplasticity. The existing `seas_spatial_dyn_driver` already supplies ~85% of what TPV26 needs; TPV27 adds **one new physics module** — an operator-split, per-node viscoplastic stress return-map applied after each elastic step. A single `if (plasticity_enabled)` makes TPV26 and TPV27 share one code path that is *bit-identical* when plasticity is off.

1.1 0. Executive summary

0.1 What already exists and is reused verbatim

- **Velocity–stress ADER-DG wave operator** (`dynamic/wave_operator.{hpp,cpp,inl}`): 9-component state $Q = [\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}, v_x, v_y, v_z]$ per nodal DOF (`dynamic/wave_state.hpp:2338`), **nodal Gauss–Lobatto L2 basis** (`wave_operator.inl:36`), homogeneous (λ, μ, ρ) flux Jacobians built once (`godunov_flux.hpp`). Identical operator for TPV26, TPV27 and TPV205 — **no operator change for plasticity** (it is operator-split post-step).
- **Closed-form LSW solver** `SolveLSW_TPV205` (`dynamic/tpv205_friction.hpp:104`) — already takes a cohesion and a `sigma_n_floor` argument; implements $\tau_{\text{strength}} = \mu \cdot \max(\sigma_n, \text{floor}) + C_0$, exactly the spec’s $\tau = C_0 + \mu \cdot \max(0, \sigma_n - P_f)$ (Part 4 p.10).
- **Forced-rupture friction coefficient** `LSWFrictionCoefficient_ForcedRupture` (`spatial/code/spatial_friction.hpp:1934`) — implements $\mu = \mu_s + (\mu_d - \mu_s) \cdot \max(f_1(\delta), f_2(t))$ *byte-for-byte* per spec Part 4/5, and reduces byte-identically to plain LSW at the $T_{\text{forced}} \geq 1e8$ sentinel (verified by reading the function).
- **Depth-linear cohesion taper** in `SpatialFrictionResolver::ResolveSlipWeakening` (`spatial/code/spatial_friction.hpp:1934`): $C_0 = \max(\text{floor}, \text{floor} + \text{grad} \cdot (\text{ref_depth} - \text{depth}))$.
- **Generic fault-stress projection** `FaultGeometry::ComputeParams<StressSource3D>` (`fault/fault_geometry.hpp:135`): projects any Cauchy-tensor source onto each fault DOF, subtracts a depth-dependent pore pressure, clamps $\min_{\sigma_n}$.
- **Off-fault surface-station + displacement accumulator** (`dynamic/tpv6_stations.hpp`, `TPV205SurfaceStationWriter`), **on-fault station writer** (`dynamic/tpv205_stations.hpp`), ADER/RK steppers, mixed flux, PML, free-surface + absorbing BC, ParaView/HDF5+ZFP, checkpoint/restart, fault-locality MPI partition — all problem-agnostic.
- **DOFData carries** `lsw_cohesion`, `T_forced_rupture` (default $1e9$), `t0_decay_forced` (default 0) (`dynamic/fault_face_flux.hpp:113–140`); `InitializeFaultDOFs_Spatial` already threads per-DOF `T_forced_s/t0_decay_s` (`dynamic/spatial_setup.hpp:266`).

0.2 What must be built (ordered by risk)

1. (TPV26+27, friction) **Wire forced-rupture nucleation end-to-end** — currently half-built and *unwired*:
 - re-add `SpatialFrictionResolver::ResolveForcedRupture` (computes per-DOF T_{forced} from the SCEC $T(r)$ formula); formula preserved under `#if 0` in `tests/unit/test_spatial_friction_r`
 - re-add `NucleationKind::ForcedRupture` + a `[nucleation]` `kind="forced_rupture"` parser.
 - flip `SetFaultFrictionLaw` to `LSW_ForcedRupture` and feed real T_{forced}/t_0 (replace the dummy `1e9/0` at `spatial_dyn_driver.cpp:1397`, 1974–1985).
 - **Fix the “round-6” interior-face bug**: extend `LinearSlipWeakeningIterator::StepOneQP_` (`friction_substep_iterator.cpp:221`) to call `LSWFrictionCoefficient_ForcedRupture` with the substep absolute time so **interior** fault QPs (the bulk of the fault) get forced rupture — today only shared/seam faces would.
2. (TPV26+27, stress) **New depth-dependent Cauchy stress source** — `Tpv2627DepthStressSource` implementing the SCEC depth profile (σ_{22} lithostatic, σ_{11}/σ_{33} via `b11/b33`, σ_{13} via `b13`, $\Omega(\text{depth})$ taper, fluid pressure), in the **codebase frame + compression-positive geology** convention so the existing projection is reused. New `StressSourceKind` + parser + one driver dispatch arm.
3. (framework) **Nonlinear bulk-rheology path** — a new, swappable `BulkRheology` constitutive interface (constitutive/) applied operator-split per node at the integrator chokepoint (`spatial_dyn_driver.cpp:3363`), selected by a `[rheology]` config block. Members: `LinearElastic` (no-op default $\Rightarrow$ TPV26 & all existing problems bit-identical), `DruckerPragerViscoplastic` (TPV27, the first nonlinear model), and a reserved seat for a future `ContinuumDamageBreakage` model (§4.4.7). This replaces the one-off plasticity bolt-on.
4. **Output** — TPV26/27 on-fault station writer (12 stations, 8 cols), off-fault surface station writer (6 body stations, 7 cols, displacement via the `tpv6` accumulator), and a **new rupture-time cplot** tracker (no rupture-time tracking exists today).
5. **Meshes + configs** — `Gmsh v2.2 .geo` for a 40 km $\times$ 20 km surface-breaking vertical fault (reparametrize `tpv31/mesh/tpv31_50m.geo`); `tpv26/configs/*.toml` and `tpv27/configs/*.toml`; `[problem]` tag dispatch lines.
6. **Tests + build wiring.**

0.3 Key design decisions (with rationale)

Decision	Choice	Why
Nonlinear rheology = separate path	A BulkRheology interface (constitutive/) with model registry, applied operator-split per node after the elastic step; wave operator stays linear-elastic	Keeps the elastic core untouched, isolates/independently-tests each nonlinear model, and lets TPV27 (Drucker–Prager) and a future CBM share one path. The linear-elastic member is a no-op $\Rightarrow$ existing problems are byte-identical (§4.4).

Decision	Choice	Why
Rheology hook granularity	Once per macro-step , operator-split, at Q.Swap chokepoint	Single site covers ADER+RK4+RK45; does not perturb the ADER predictor or fault Riemann solve; dt_step/t in scope (spatial_dyn_driver.cpp:3335–3366). Consistent with the spec’s per-step return map (SeisSol uses the same cadence).
Rheology math home	New constitutive/{bulk_rheology.hpp, linear_elastic_rheology.hpp, drucker_prager_core.{hpp,cpp}, drucker_prager_rheology.{hpp,cpp}, bulk_rheology_factory.{hpp,cpp}.path. [rev-5] The DP <i>math</i> is distinct from the QD ConstitutiveModel	The dynamic wave operator never calls ConstitutiveModel ($\varepsilon \rightarrow \sigma$, QD-only). The new BulkRheology is the stress-based, operator-split analogue for the velocity-stress path. [rev-5] The DP <i>math</i> is split into an embedding-agnostic core (drucker_prager_core) so a future QD off-fault-plasticity adapter reuses it (§4.4.8); the dynamic model is a thin adapter over the core.
Total-stress reconstruction	Store bulk σ^0_eff (effective, tension-positive) per DOF; form $s = \sigma^0_eff + Q[0..5]$ inside the kernel; apply the spec Part-6 return map and write back the deviatoric change	Driver runs fluctuation-Q ($Q_bg=0$); the spec yield needs total stress. Storing the <i>effective</i> initial stress (Pf folded into the diagonal) so the yield drops the explicit Pf term is the spec’s own Part-7 p.20 remark (SeisSol uses the same option — a corroborating reference, not a parity target). Mean stress is unchanged by the return map $\Rightarrow$ the write-back is a pure deviatoric decrement (see §4.4.3).

Decision	Choice	Why
Nucleation	Forced rupture (time-weakening) , NOT overstress	Spec is explicit (Part 1 p.3): nucleation must NOT raise near-hypocenter shear stress (inconsistent with the prescribed volumetric stress); use the forced-rupture zone. gradual overstress retained only as a non-faithful smoke fallback.
Initial-stress storage method	Method 1 (stress-change / fluctuation-Q)	Matches the driver's existing $Q=0$ + side-channel pre-stress design; spec Part 7 endorses it; no explicit gravity body force or balancing boundary tractions needed. TPV27 stores σ^0 per bulk node (Method-1 requires it for inelastic).
Mesh	One mesh for both; reparametrize tpv31_50m.geo	TPV27 plasticity is a constitutive change only; geometry/tags identical. tpv31 template is the cleanest (uniform fault refinement, surface-breaking embedding, minimal size field).

1.2 1. Benchmark specification (condensed, verified)

All values cross-checked by two independent PDF extractions + an adversarial verification pass that re-derived the consistency numbers. **Spec frame:** x = along-strike, y = depth (positive downward), z = fault-perpendicular; fault is the plane $z=0$; compression is **negative** (standard continuum / tension-positive sign).

1.1 Geometry & material (shared)

- Single planar **vertical right-lateral strike-slip** fault, **40 km along-strike** ($-20000 \leq x \leq 20000$) $\times$ **20 km down-dip** ($0 \leq y \leq 20000$), **surface-breaking** (top edge at the free surface $y=0$).
- Hypocenter $(x, y, z) = (-5000, 10000, 0)$ (15 km from left edge, 10 km deep).
- Fault BC: slip $\rightarrow 0$ at the fault border; node exactly on border may not slip; the **free surface is NOT a border**; fault may not open (slide-parallel only under tension; keep accumulating path-integrated slip).
- Homogeneous half-space: $\rho=2670$, $V_s=3464$, $V_p=6000 \Rightarrow \mu=\rho V_s^2 \approx 3.2038e10$, $\lambda=\rho(V_p^2-2V_s^2) \approx 3.2044e10$ (Poisson 0.25), $K=\lambda+2/3\mu \approx 5.34e10$. **Same moduli for TPV26 and TPV27.**

- $g = 9.8$ exactly. Fluid pressure $P_f = 1000 \cdot 9.8 \cdot \text{depth}$ (hydrostatic, constant in time).

1.2 Depth-dependent initial stress tensor (shared; spec Part 3, compression-negative)

$\text{depth} = y$ (spec) $P_f = 1000 \cdot 9.8 \cdot \text{depth}$
 $\sigma_{22} = -2670 \cdot 9.8 \cdot \text{depth}$ (vertical/lithostatic, intermediate principal axis)
 $\Omega(\text{depth}) = 1$ if $\text{depth} \leq 15000$
 $\quad = (20000 - \text{depth})/5000$ if $15000 \leq \text{depth} \leq 20000$
 $\quad = 0$ if $\text{depth} \geq 20000$
 $\sigma_{11} = \Omega \cdot (b_{11} \cdot (\sigma_{22} + P_f) - P_f) + (1 - \Omega) \cdot \sigma_{22}$ (fault-parallel)
 $\sigma_{33} = \Omega \cdot (b_{33} \cdot (\sigma_{22} + P_f) - P_f) + (1 - \Omega) \cdot \sigma_{22}$ (fault-perpendicular; $-\sigma_{33}$ = total normal on fault)
 $\sigma_{13} = \Omega \cdot (b_{13} \cdot (\sigma_{22} + P_f))$ (on-fault shear; + = right-lateral)
 $\sigma_{12} = \sigma_{23} = 0$
 $b_{11} = 0.926793$ $b_{33} = 1.073206$ $b_{13} = -0.169029$
 effective stress = total + P_f on the three diagonal terms

Verified consistency checks (golden unit-test targets) at hypocenter depth 10 km ($\Omega=1$): - total normal on fault $-\sigma_{33} = 273.64$ MPa; effective normal $\sigma_{n_eff} = 175.64$ MPa; on-fault shear $\sigma_{13} = 27.66$ MPa; ratio $27.66/175.64 = 0.1575$ (strictly between $\mu_d=0.12$ and $\mu_s=0.18$). - DP (TPV27): $\sqrt{J_2} = 30.15$ MPa, yield $Y = 32.40$ MPa, $\sqrt{J_2}/Y = 0.930 \Rightarrow$ the spec's stated "93 % of yield".

1.3 Friction (linear slip-weakening; shared; spec Part 4)

- $\mu = \mu_s + (\mu_d - \mu_s) \cdot \max(f_1, f_2)$, $\mu_s=0.18$, $\mu_d=0.12$, $d_0=0.30$ m.
- $f_1 = \min(D/d_0, 1)$ where **D** is **path-integrated slip** ($\int |V| dt$, not net displacement — worked example p.10: 0.4 m forward then 0.1 m back $\Rightarrow D=0.5$ m).
- $f_2 = 0$ ($t < T$); $(t-T)/t_0$ ($T \leq t < T+t_0$); 1 ($t \geq T+t_0$), $t_0 = 0.5$ s.
- Sliding strength $\tau = C_0 + \mu \cdot \max(0, \sigma_n - P_f)$.
- **Frictional cohesion** $C_0 = 0.40$ MPa + 0.00072 MPa/m $\cdot (5000 - \text{depth})$ for $\text{depth} \leq 5000$, else 0.40 MPa $\Rightarrow 4.0$ MPa at surface, 0.40 MPa below 5 km (suppresses free-surface effects). (Distinct from the DP bulk cohesion c .)

1.4 Nucleation (forced rupture; shared; spec Part 5)

- $T(r) = r/(0.7 \cdot V_s) + (0.081 \cdot r_{crit}/(0.7 \cdot V_s)) \cdot (1/(1-(r/r_{crit})^2) - 1)$ for $r < r_{crit}$; else $1e9$.
- r = distance to hypocenter in the fault plane; $r_{crit} = 4000$ m; $T(0)=0$. Forced-rupture front speed decreases from $0.7 \cdot V_s$ near the hypocenter to 0 at r_{crit} .

1.5 TPV27 off-fault Drucker-Prager viscoplasticity (TPV27 only; SCEC spec Part 6 is authoritative)

- Bulk cohesion $c = 1.36$ MPa; bulk friction $\nu = 0.1934 \Rightarrow \varphi = \text{atan}(\nu) = 0.19105$ rad $\approx 10.95^\circ$ ($\cos \varphi = 0.98182$, $\sin \varphi = 0.18983$); viscoplastic relaxation time $T_v = 0.03$ s.
- **Spec return map (Part 6 p.13-15 — implement this)**: total stress $\sigma = \sigma^0 + \Delta \sigma$ (initial + elastic change); $\sigma_m = (\sigma_{11} + \sigma_{22} + \sigma_{33})/3$; deviator $s_{ij} = \sigma_{ij} - \sigma_m \cdot \delta_{ij}$; $J_2 = 1/2 \sum s_{ij} s_{ij}$; $\tau = \sqrt{J_2}$; yield $Y = \max(0, c \cdot \cos \varphi - (\sigma_m + P_f) \cdot \sin \varphi)$, $F = \sqrt{J_2} - Y$; if $F \leq 0$ elastic (no change); else $r = \exp(-dt/T_v) + (1 - \exp(-dt/T_v)) \cdot Y / \sqrt{J_2}$, and $\sigma_{new} = \sigma_m \cdot \delta$

+ $r \cdot s$ (mean unchanged; deviator relaxed toward the yield surface). The reference Fortran (spec p.16-17: $\text{decay}=\exp(-dt/Tv)$, $\text{yldfac}=\text{decay}+(1-\text{decay}) \cdot \text{taulim}/\text{tau}$, scale deviator by yldfac) is the byte-level acceptance oracle.

- **Effective-stress storage (spec Part-7 p.20 remark — adopted):** store σ^0 as the *effective* tensor (= total + Pf on the diagonal); then $\sigma_m + \text{Pf}$ in the yield becomes just the effective mean, so the kernel needs no separate per-node Pf term. This also matches the on-fault path (already effective via the pore-pressure gradient). (*SeisSol uses this same option — a corroborating reference, not a parity target.*)
- **Plastic / viscoplastic strain** (spec Part 6 Duvaut-Lions, eq. 10): the deviatoric viscoplastic strain rate $d\varepsilon^{\text{vp}}_{ij}/dt = (\sigma_{ij} - P_{ij}(\sigma))/(2\mu \cdot Tv)$; accumulate the scalar equivalent $\eta += dt \cdot \sqrt{0.5 \sum (d\varepsilon^{\text{vp}}_{ij}/dt)^2}$ if a diagnostic is wanted. **The v13 doc requests NO plastic-strain output**, so η (and the 6 ε_{ij}) are optional diagnostic fields only.
- **Cadence:** apply the return map **once per macro time step, after the elastic (flux) update**, with the full dt — the operator-split cadence at our Q.Swap chokepoint (§2.2), consistent with the spec’s per-step “Step 1...Step 4” description.

1.6 Run & outputs (shared; spec Parts 3, 8-10)

- Run 0 $\rightarrow$ 13.0 s; submit at **100 m and 50 m** fault node spacing (50 m preferred).
- **On-fault** (12 stations, ASCII, 8 cols): t, h-slip, h-slip-rate, h-shear-stress, v-slip, v-slip-rate, v-shear-stress, n-stress. h = along-strike (+ = right-lateral); v = down-dip (+ = far/+z side down); n-stress = **effective** normal ($\sigma_n - \text{Pf}$, + = extension). Stress in MPa, slip in m, rate in m/s.
 - Station list (along-strike km, depth km): faultst-050dp000 (-5,0), faultst150dp000 (15,0), faultst-050dp050 (-5,5), faultst150dp050 (15,5), faultst-150dp100 (-15,10), faultst-050dp100 (-5,10)=HYPOCENTER, faultst000dp100 (0,10), faultst050dp100 (5,10), faultst100dp100 (10,10), faultst150dp100 (15,10), faultst-050dp150 (-5,15), faultst150dp150 (15,15).
- **Off-fault** (6 stations, all at the free surface, ASCII, 7 cols): t, h-disp, h-vel, v-disp, v-vel, n-disp, n-vel. h = +x (strike), v = +y (down), n = +z (toward far side). Disp m, vel m/s.
 - Station list bodyAAAstBBBdpCCC (AAA = signed $\perp$ -distance/100 m, + = far/+z; BBB = strike/100 m; depth 0): body030st-050dp000 (-5km, z=+3000), body-030st-050dp000 (-5, z=-3000), body030st050dp000 (5, +3000), body-030st050dp000 (5, -3000), body030st150dp000 (15, +3000), body-030st150dp000 (15, -3000).
- **cplot** (one file): j (=x, -20000..20000), k (=depth, 0..20000), t = first time slip-rate exceeds **0.001 m/s** (else 1e9).
- **Header tokens (verbatim for the SCEC server):** # problem=, # author=, # date=, # code=, # code_version=, # element_size=, # time_step=, # num_time_steps=, # location=, then an unprefixed field-list line. **cplot uses a reduced header** (omit time_step/num_time_steps/location). Recommended numeric formats: data 14.6E, **time 20.12E** (high-precision time required for the time-series so the server’s equal-step digital-filter check passes; cplot needs only 14.6E).
- **Output cadence is decoupled from solver dt:** the explicit ADER/Godunov CFL dt must be **resampled onto a uniform output grid** (e.g. 0.005 s or 0.008 s) with equal steps; the example

uses `time_step=0.008`, `num_time_steps=1625` (= 13.0 s).

1.3 2. Codebase architecture relevant to TPV26/27

2.1 Driver flow (drivers/spatial_dyn_driver.cpp, ~3724 lines)

CLI parse → LoadSpatialFrictionConfig (TOML) → CLI overrides → `is_lsw` from `cfg.law` → mesh load (:986) → fault-locality partition (:992) → MaterialField (Constant mode for homogeneous) → WaveOperator ctor (:1246) → **SetFaultFrictionLaw (:1397)** → per-DOF fault tables → FaultGeometry → **stress seeding (:1683–1806)** → friction resolve (:1813) → **nucleation (:1878)** → FaultFaceFlux (:1955) → **InitializeFaultDOFs_Spatial (:1982, fed dummy `T_forced=1e9`)** → friction iterator (:3004) → **time loop (:3311)** → output/checkpoint.

The **[problem] tag is physics-inert** — it only selects the on-fault station writer at :3294–3299. All physics is keyed on `cfg.law`, `cfg.stress.kind`, `cfg.nucleation.kind`.

2.2 The integrator chokepoint (single bulk-rheology hook site) – verified

```
:3335 if (is_rk && is_lsw) AdvanceRKCoupledLSW_Spatial(...) // Q_new
:3349 else if (is_rk) AdvanceRKCoupled_Spatial(...) // Q_new
:3359 else AdvanceADERWithSubStep_Spatial(...) // Q_new (default)
:3364 Q.Swap(Q_new); t += dt_step; last_completed_step = step+1;
```

`dt_step` and `t` are in scope. The **bulk-rheology update `rheology->ApplyStep(ctx, dt_step, Q_new, state)` (§4.4) goes immediately before `Q.Swap(Q_new)`** — one site covers all three integrators; skipped entirely for the linear-elastic no-op. (*Forced rupture itself requires the ADER substep path; the RK LSW path uses `EvaluateLSW` and aborts on `LSW_ForcedRupture`. TPV26/27 therefore run with the default `--time-integrator ader`.)*

2.3 Stress representation – verified

- `Q` is **fluctuation** stress, tension-positive (`wave_state.hpp:26` “Matches SeisSol ordering”; `EnergyDensity` uses standard tension-positive elasticity). Bulk starts at rest (`Q=0`, `spatial_dyn_driver.cpp:2932`); `Q_bg=0` (:2012–2016). Initial loading enters **only at the fault** (`DOFData tau_pre/sigma_n`). There is **no per-DOF bulk initial-stress field** today.
- `StressSource3D` (`spatial/code/spatial_stress.hpp:11–29`) returns a 3×3 Cauchy tensor in **compression-positive geology** convention (CLAUDE.md: “ $\sigma_n > 0$ = compression”). `ComputeParams` projects it to per-DOF (σ_n , τ_{dip} , τ_{strike}) and subtracts a depth-dependent pore pressure.

2.4 Coordinate frames & sign conventions – MASTER TABLE (load-bearing)

Axis (physical)	Spec frame	Codebase frame
Along-strike	x (axis 1)	x
Vertical / depth	y (axis 2, down +)	−z (depth = −z_code, z<0 is down; up=(0,0,1))
Fault-perpendicular	z (axis 3)	y (fault plane y=0, ref_normal=(0,−1,0))

Axis (physical)	Spec frame	Codebase frame
Hypocenter	(-5000, 10000, 0)	(x=-5000, y=0, z=-10000)

Stress-tensor remap spec $\rightarrow$ codebase (diagonal terms invariant; the only nonzero shear σ_{13} maps to the **code xy** pair because spec-axis-1=strike $\rightarrow$ code-x and spec-axis-3=normal $\rightarrow$ code-y):

$$\begin{aligned}\sigma_{xx}(\text{code}) &= \sigma_{11}(\text{strike}) & \sigma_{yy}(\text{code}) &= \sigma_{33}(\text{fault-normal}) & \sigma_{zz}(\text{code}) &= \sigma_{22}(\text{vertical}) \\ \sigma_{xy}(\text{code}) &= \sigma_{13}(\text{on-fault shear}) & \sigma_{yz}(\text{code}) &= 0 & \sigma_{xz}(\text{code}) &= 0\end{aligned}$$

On the fault ($n = \text{code-y}$): $\sigma_n = \sigma_{yy} = \sigma_{33}$ ($\checkmark$ total normal = $-\sigma_{33}$ once compression-positive), on-fault strike shear = $\sigma_{xy} = \sigma_{13}$ ($\checkmark$ right-lateral). The driver already negates σ_{xy} at source construction (`spatial_dyn_driver.cpp:1699-1704`) to keep right-lateral-positive — the new source must be written so that, after that existing negation, the projected τ_{strike} is right-lateral-positive (validate by unit test, do not assume).

Sign conventions to honor (CLAUDE.md + verified): - 0 stress: **tension-positive**. `StressSource3D`: **compression-positive geology**. $\Rightarrow$ bulk σ^0 for plasticity = $-(\text{StressSource tensor})$ (negate to tension-positive). The spec's stress tensor is *already* tension-positive (compressive σ_{22} is negative), so $\sigma^0_{\text{tension-positive}} = \text{spec values}$. - Fault-local frame (CLAUDE.md): `tangent1 = dip`, `tangent2 = strike`; for this vertical fault `can_t1=(0,0,-1)` (down-dip), `can_t2=(+1,0,0)` (strike). In `D0FData`, channel 1 = dip, channel 2 = strike. - Effective normal: `configure [pore_pressure]` `P_p_grad_pa_per_m = 9800` so `ComputeParams` yields $\sigma_{n_eff} = \sigma_n - 1000 \cdot 9.8 \cdot \text{depth}$ ($= \sigma_n - P_f$), exactly the spec's reported “n-stress”.

2.5 Why ConstitutiveModel is the wrong hook

`constitutive/{constitutive_model.hpp, linear_elastic.hpp}` ($\varepsilon \rightarrow \sigma$, `UpdateState`=“return mapping”) is used **only** by the quasi-dynamic displacement operator (`domain/elasticity_operator.hpp`, `integrator/dg_elasticity_*`) — confirmed by `grep` returning zero references in `dynamic/`. The dynamic DP correction is a **stress $\rightarrow$ stress nodal projection on Q** that needs neither ε nor C_{tang} .

1.4 3. Gap analysis

Capability	TPV26	TPV27	Status
Velocity-stress	$\checkmark$	$\checkmark$	reuse verbatim
ADER-DG operator			
LSW closed-form solve	$\checkmark$	$\checkmark$	reuse verbatim
+ cohesion + σ_n			(<code>tpv205_friction.hpp:104</code>)
floor			
Forced-rupture $\mu(\delta, t)$	$\checkmark$	$\checkmark$	exists
coefficient			(<code>spatial_friction.hpp:813</code>), unwired
Depth-linear cohesion	$\checkmark$	$\checkmark$	config-only
$C_0(z)$			(<code>ResolveSlipWeakening</code>)

Capability	TPV26	TPV27	Status
Depth-dependent	✓	✓	NEW source class + kind
Cauchy fault prestress			
ResolveForcedRupture (per-DOF $T(r)$)	✓	✓	NEW (was removed; formula in #if 0 tests)
Interior-face forced rupture (round-6 fix)	✓	✓	NEW iterator branch
Driver flip to LSW_ForcedRupture + real T_{forced}	✓	✓	MODIFY (:1397, :1974–1985)
BulkRheology interface + factory + LinearElastic no-op + driver hook	✓ (no-op)	✓	NEW framework (constitutive/, [rheology] config, hook :3363)
Bulk per-DOF effective σ^0_{eff} field	—	✓	NEW (model Setup)
DruckerPragerViscoplastic return-map model		✓	NEW BulkRheology member
On-fault station writer (12 stations, 8 col)	✓	✓	NEW (clone tpv205_stations.hpp)
Off-fault surface stations + displacement	✓	✓	NEW list , reuse SurfaceStationWriter/tpv6 accumulator
cplot rupture-time	✓	✓	NEW tracker (none exists)
Mesh (40×20 km surface-breaking vertical)	✓	✓	NEW .geo (reparam tpv31_50m.geo)
Configs + [problem] tag dispatch	✓	✓	NEW TOMLs + 2 dispatch lines

1.5 4. Detailed design

4.1 Depth-dependent initial stress source (TPV26 + TPV27)

New class `spatial::Tpv2627DepthStressSource` in `spatial/code/spatial_stress.{hpp,cpp}` satisfying the `StressSource3D` concept (`Evaluate(x,y,z)→DenseMatrix(3,3)`, `BBox`, `ContainsBBox`), modeled on `DepthProportionalToShearModulusStressSource`.

- `Evaluate(x,y,z)`: $\text{depth} = \max(0, -z)$; compute P_f , σ_{22} , Ω , σ_{11} , σ_{33} , σ_{13} per §1.2; assemble the 3×3 Cauchy tensor in the **codebase frame** using the §2.4 remap, returned in **compression-positive geology** convention (negate the tension-positive spec values) so `ComputeParams` and the existing σ_{xy} negation produce a right-lateral-positive τ_{strike} and a compression-positive σ_n .

- Config params (so the class is not hard-coded): rho=2670, g=9.8, b11, b33, b13, omega_top_m=15000, omega_bot_m=20000, water_density=1000.

New StressSourceKind::Tpv2627Depth + string "tpv2627_depth": - spatial/code/spatial_friction.hpp:224 (enum) + StressSpec fields. - spatial/code/spatial_friction.cpp:423–431 (parse_stress_kind) + :941–1015 (validation). - drivers/spatial_dyn_driver.cpp:1683–1806 — add a dispatch arm constructing the source and calling geom.ComputeParams<Tpv2627DepthStressSource>(src, P_p=0, P_p_grad=9800, min_sigma_n=0). The **same source object** is queried at bulk DOFs to build σ^0 for TPV27 (§4.5).

Effective normal at the surface-breaking top node (spec Part 7 p.19): the pore-pressure-gradient path gives $P_f=0$ exactly at $z=0$, but spec says a surface fault node should use P_f for depth = 1/2 element (uniform stress) or 1/3 element (linear). Near the surface $C_0 = 4$ MPa dominates and the spec states this cohesion exists to “suppress free-surface effects”, so $P_f=0$ at the very top node is likely acceptable. **Open verification item** (§7): if the top node misbehaves, add a small min_sigma_n or a half-element P_f offset on the top row.

4.2 Forced-rupture nucleation (TPV26 + TPV27)

(a) **SpatialFrictionResolver::ResolveForcedRupture** (re-add to spatial/code/spatial_friction.{hpp,cpp}): for each fault DOF at code $(x, y \approx 0, z)$, $r = \sqrt{(x - x_{\text{hyp}})^2 + (z - z_{\text{hyp}})^2}$ with hypocenter $(x_{\text{hyp}}=-5000, z_{\text{hyp}}=-10000)$; $T_{\text{forced}} = T(r)$ per §1.4 ($1e9$ for $r \geq r_{\text{crit}}$); $t_0_{\text{decay}} = t_0$ uniform. Returns $\{T_{\text{forced}_s}, t_0_{\text{decay}_s}\}$ vectors. **The exact $T(r)$ and its unit-test values are preserved under #if 0 in tests/unit/test_spatial_friction_resolver.cpp:890–1113** — revive both impl and tests.

(b) **NucleationKind::ForcedRupture** + [nucleation] kind="forced_rupture" parser (spatial_friction.hpp:43 enum + the [nucleation] parse path), sub-block: hypocenter_x/y/z, rcrit_m=4000, vs=3464, vr_factor=0.7, t0_s=0.5.

(c) **Driver wiring** (drivers/spatial_dyn_driver.cpp): - :1397 SetFaultFrictionLaw: select FaultFrictionLaw::LSW_ForcedRupture when the nucleation kind is forced_rupture. - :1974–1985: replace dummy_T_forced=1e9/dummy_t0_decay=0 with ResolveForcedRupture(...) output passed into InitializeFaultDOFs_Spatial (:1982).

(d) **Interior-face forced rupture — the “round-6” fix** (the trickiest change; byte-exact-sensitive file): LinearSlipWeakeningIterator::StepOneQP_ (dynamic/friction_substep_iterator.cpp:221) currently calls LSWFrictionCoefficient_TPV205(delta, μ_s , μ_d , d_c) with **no time term**. The wave operator’s inline EvaluateADER_LSW_ForcedRupture covers only shared/seam faces, so interior fault QPs (including the hypocenter) would silently get plain LSW. - Thread the **substep absolute end-time** t_{sub} (available at the call site ~:317) into StepOneQP_. - When the active law is LSW_ForcedRupture (gate on flux_.WaveOpLaw()), call LSWFrictionCoefficient_ForcedRupture(delta, μ_s , μ_d , d_c, t_{sub} , d.T_forced_rupture, d.t0_decay_forced). - **Byte-exact contract:** keep the plain-LSW (TPV205) call path literally unchanged. LSWFrictionCoefficient_ForcedRupture reduces byte-identically to the plain formula at $T_{\text{forced}} \geq 1e8$ (verified), so a regression test must show TPV205 traces are unchanged. - Add a MakeFrictionIterator branch (dynamic/friction_iterator_factory selecting the forced-rupture-aware iterator. Ensure interior (iterator) and seam (inline EvaluateADER_LSW_ForcedRupture) paths use **identical** $\mu(\delta, t)$ to avoid a seam discontinuity in MPI.

4.3 Depth-dependent frictional cohesion (TPV26 + TPV27, config-only)

Set in [friction.slip_weakening]: cohesion_grad_pa_per_m=720, cohesion_ref_depth_m=5000, cohesion_floor_pa=0.40e6, cohesion_taper_axis="z" (depth = $\max(0, -z)$). The resolver's $C0 = \max(\text{floor}, \text{floor} + \text{grad} \cdot (\text{ref_depth} - \text{depth}))$ yields $C0(0)=4.0$ MPa, $C0(5000)=0.40$ MPa, $C0(\geq 5000)=0.40$ MPa — exactly the spec. **No new code.**

4.4 Nonlinear bulk rheology – a separate, extensible constitutive path

Design intent (per the architecture goal). The codebase's dynamic path is currently *only* linear elastic. Rather than bolt Drucker–Prager onto the driver as a one-off, TPV27 is the occasion to add a **clean, swappable nonlinear-rheology path**: one interface, multiple bulk constitutive models, selected by config. The linear-elastic case becomes the trivial (no-op) member, Drucker–Prager viscoplasticity is the first nonlinear member (this deliverable), and the interface is **deliberately shaped so a future continuum damage–breakage model (CBM) drops in** without touching the driver or the wave operator's elastic core (§4.4.7).

1.5.4.1 4.4.1 Architecture & placement

- The **WaveOperator** stays **purely linear-elastic** (velocity–stress DG, constant flux Jacobians). All nonlinear constitutive behavior is an **operator-split, per-node “bulk material update”** applied once per macro step, *after* the elastic + fault-flux update, at the single chokepoint (spatial_dyn_driver.cpp:3363, before Q.Swap). This keeps the hot elastic kernels untouched and every nonlinear model isolated and independently testable.
- Home: a new family under **constitutive/** — the natural home for constitutive relations — kept **distinct from the existing ConstitutiveModel** (which is a strain→stress interface used only by the quasi-dynamic *displacement* operator, §2.5). The new interface is the **stress-based, operator-split analogue for the dynamic velocity–stress path**. (A future unification of the two is possible but explicitly out of scope.)
- Valid because the basis is **nodal GLL** (wave_operator.inl:36): each DOF value is the physical nodal field value, so the update is a flat pointwise loop over ndof_total_ — no quadrature, no assembly, no MPI.

```
namespace mfem::seas {

// Operator-split nonlinear bulk constitutive update for the dynamic velocity–stress DG path.
// Applied ONCE per macro step, AFTER the elastic+flux update, pointwise at each nodal GLL DOF.
class BulkRheology {
public:
    struct Context {                // immutable per-run data a model may need
        int          ndof_total;    // # nodal DOFs (component-major stride of Q)
        const real_t* node_xyz;     // 3*ndof_total physical coords (built once by the driver)
        real_t        lambda, mu, rho; // background linear moduli (homogeneous half-space)
        const real_t* sigma0_eff;   // 6*ndof effective initial stress, INJECTED by the driver (nu
        // TPV27 = Phase-1 Tpv2627DepthStressSource sampled at bulk
```

```

// full cycle = C:eps(u_QD) from the interseismic solve (§4.
};
virtual ~BulkRheology() = default;

// # per-node internal state variables: 0 (linear elastic) | 1 or 7 (Drucker–Prager) | ~2–3 (CBM)
virtual int NumStateVars() const = 0;

// One-time setup: precompute per-node model fields (e.g. effective sigma0, initial damage); zero
virtual void Setup(const Context& ctx, Vector& state /*NumStateVars()*ndof_total, in/out*/) = 0;

// Apply the operator-split update to Q's 6 stress components (+ `state`), advancing by dt.
// Returns # active/yielding nodes (diagnostic). MUST be a bit-exact no-op for LinearElastic.
virtual std::size_t ApplyStep(const Context& ctx, real_t dt, Vector& Q, Vector& state) = 0;

// ParaView names of the NumStateVars() internal fields (e.g. {"plastic_strain"} or {"alpha","br
virtual std::vector<std::string> StateFieldNames() const { return {}; }

// FORWARD-COMPAT (CBM, §4.4.7): per-node effective moduli for the NEXT elastic step, or nullptr
// moduli are constant (linear elastic, plasticity). Non-null ⇒ driver routes them into the wave
// heterogeneous flux path. Returns nullptr in this deliverable.
virtual const Vector* EffectiveModuli(const Context& ctx) const { return nullptr; }

virtual std::string Name() const = 0;
};

std::unique_ptr<BulkRheology> MakeBulkRheology(const RheologyConfig& cfg); // factory

} // namespace mfem::seas

```

1.5.4.2 4.4.2 The BulkRheology interface (the clean path) — constitutive/bulk_rheology.hpp

Driver integration (drivers/spatial_dyn_driver.cpp): - After the wave operator + geometry are built, compute per-DOF node_xyz once (bulk analogue of the fault-QP coord walk at :278–337), construct rheology = MakeBulkRheology(cfg.rheology), allocate Vector rheo_state(rheology->NumStateVars()*ndof_total), and call rheology->Setup(ctx, rheo_state). - At the chokepoint (:3363, before Q.Swap(Q_new)): if (rheology->NumStateVars() > 0) rheology->ApplyStep(ctx, dt_step, Q_new, rheo_state); — covers ADER + both RK paths in one site. - **Byte-identity guarantee:** kind="linear_elastic" ⇒ NumStateVars()==0 ⇒ the call is skipped entirely ⇒ TPV26 (and every existing problem) is bit-identical to today. - **Checkpoint/restart:** extend Read/WriteCheckpoint (:2941) to persist the opaque rheo_state (generic, sized by NumStateVars()); per-node precomputed fields (e.g. sigma0_eff) are rebuilt in Setup on restart, not checkpointed. - **Output:** register rheology->StateFieldNames() as ParaView domain fields and copy from rheo_state each cycle, mirroring the bulk stress block (:2748–2765, :3158–3175).

Config — replaces the old [plasticity] block with an extensible [rheology]:

```

[rheology]
kind = "linear_elastic"           # default (TPV26 & all existing problems); no-op
# kind = "drucker_prager"        # TPV27
# kind = "continuum_damage_breakage" # FUTURE (§4.4.7)

[rheology.drucker_prager]        # parsed only when kind = "drucker_prager"
cohesion_pa = 1.36e6
bulk_friction = 0.1934           # phi = atan(bulk_friction)
Tv_s = 0.03
output_state = true              # emit eta (+ optional ep_ij) ParaView fields
# Pf for sigma0_eff reuses [stress].water_density / [stress].g (not duplicated here)

```

RheologyConfig (kind enum + per-kind sub-struct) is parsed in `spatial/code/spatial_friction.{hpp,cpp}` alongside the other blocks.

1.5.4.3 4.4.3 LinearElasticRheology (default; TPV26 and all existing problems) — constitutive/linear_elastic_rheology.hpp NumStateVars() $\Rightarrow$ 0; Setup and ApplyStep are empty; EffectiveModuli returns nullptr; Name() $\Rightarrow$ "linear_elastic". The driver never invokes it (gated on NumStateVars() $\Rightarrow$ 0), so the path is provably zero-overhead and byte-exact.

1.5.4.4 4.4.4 DruckerPragerViscoplastic (first nonlinear model; TPV27) — thin dynamic adapter over a shared core

[rev-5] Embedding-agnostic core + adapter (per user direction; off-fault plasticity is wanted in the QD path later, §4.4.8). The DP *math* — yield function, Duvaut-Lions return map, state-rate evolution, effective- σ^0 assembly — lives in a **stateless, embedding-agnostic core** `constitutive/drucker_prager_core.{hpp,cpp}`, tensor-in/tensor-out, owning no state and no discretization. `DruckerPragerViscoplastic` (`constitutive/drucker_prager_rheology.{hpp,cpp}`) is the **thin dynamic adapter**: it presents the BulkRheology operator-split face over Q, calling the core per node. A future **QD adapter** (`ConstitutiveModel`, $\varepsilon \rightarrow \sigma$) wraps the *same* core, adding only the consistent algorithmic tangent the implicit DG solve needs (the explicit dynamic path never needs it). This is why the return map is regime-agnostic: it is rate-*dependent* ($r = 1 - \exp(-dt/Tv)$), so $dt \sim Tv$ (coseismic) relaxes partially and $dt \gg Tv$ (interseismic) drives $r \rightarrow 1$ — the correct slow-loading viscoplastic limit — with **no kernel change**.

The core function (embedding-agnostic; no state, no Q, no MFEM discretization types):

```

namespace mfem::seas::dp_core {
    struct Params { real_t c, cos_phi, sin_phi, Tv; };           // precomputed from cohesion, bulk
    // In: effective TOTAL stress s_eff[6] (tension-positive); dt. Out: relaxed stress + eta increme
    // Returns true if the node yielded (F>0). Pure function of its arguments.
    bool ReturnMap(const real_t s_eff[6], real_t dt, const Params& p,
                   real_t s_out[6], real_t& d_eta /*+ optional real_t d_ep[6]*//);
}

```

The **dynamic adapter** DruckerPragerViscoplastic implements BulkRheology: NumStateVars() $\Rightarrow$ 1 (the equivalent plastic strain η ; optionally 7 to also carry ϵ_{p_ij}); StateFieldNames() $\Rightarrow$ {"plastic_strain"[, "ep_xx", ..., "ep_xz"]}; EffectiveModuli returns nullptr (DP does not change moduli). Setup caches the **injected** ctx.sigma0_eff (§4.4.5) and zeros η .

ApplyStep = the per-node loop over Q; each node forms $s_eff = \text{sigma0_eff} + Q_stress$, calls `dp_core::ReturnMap`, and writes back the deviatoric change + η increment. The core body is the **SCEC spec Part-6 return map** (the authority; SeisSol agrees). Precompute once per step $r = 1 - \exp(-dt/T_v)$ (use `-std::expm1(-dt/T_v)` for small- dt/T_v accuracy; $T_v \leq 0 \Rightarrow r = 1$, rate-independent). For each bulk DOF j (stride $Q[c*ndof_total + j]$, $c \in \{SXX..SXZ\}$):

```
s[6] = sigma0_eff[6][j] + Q_stress[6][j]           // effective TOTAL stress (tension-positive)
m    = (s_xx + s_yy + s_zz)/3                     // effective mean ( $\sigma^0$  is effective  $\Rightarrow$  no extra Pf)
s_ij -= m on the three diagonal comps             // deviator
J2    = 0.5*(s_xx2 + s_yy2 + s_zz2) + s_xy2 + s_yz2 + s_xz2;  $\tau = \text{sqrt}(J2)$ 
Y     = max(0, c*cos_phi - m*sin_phi)              // spec yield;  $m < 0$  (compression)  $\Rightarrow$  strengthens
if ( $\tau > Y$ ):                                     //  $F = \tau - Y > 0$  ( $\tau > Y \geq 0 \Rightarrow \tau > 0$ , division safe)
    yieldFactor = (Y/ $\tau$  - 1.0) * r                 // spec:  $\sigma\_new = m \cdot \delta + r \cdot s \Rightarrow \Delta Q\_dev = (r-1) \cdot s = \text{yield}$ 
    for k in 6:
        upd = yieldFactor * s_ij[k]; Q_stress[k][j] += upd // STRESS UPDATE (deviatoric; mean preserved)
        dEpDot = -upd / (2*mu * T_v * r)             // spec Duvaut-Lions  $d\epsilon^{vp\_ij}/dt$ 
        if (ep) ep[k][j] += dt * dEpDot
        acc += dEpDot*dEpDot
    if (eta) eta[j] += dt * sqrt(0.5*acc)             // equivalent plastic strain (diagnostic)
// else: no change  $\Rightarrow$  TPV26 / below-yield bit-identical
```

- **Effective-stress storage** (spec Part-7 p.20 remark): $\text{sigma0_eff} = \text{total} + P_f \cdot \delta \Rightarrow$ the yield needs no separate P_f term. Same effective convention the on-fault path uses (total $- P_f$ via the pore-pressure gradient) $\Rightarrow$ fault and bulk consistent.
- **Applied at all bulk DOFs** including fault-adjacent (SCEC TPV27 plastic strain reaches the fault; do not carve an exclusion corridor). Pointwise/element-local $\Rightarrow$ **no CFL change, no MPI exchange**.
- The reference Fortran (spec p.16-17: $\text{decay} = \exp(-dt/T_v)$, $\text{yldfac} = \text{decay} + (1 - \text{decay}) \cdot \text{taulim}/\text{tau}$, scale deviator) is the byte-level acceptance oracle (§6.1). SeisSol Plasticity.cpp is a corroborating cross-check, not a parity target.

1.5.4.5 4.4.5 Per-DOF effective initial stress sigma0_eff (INJECTED, not model-built) [rev-5] sigma0_eff is **injected via `ctx.sigma0\_eff`**, not hard-wired in the model — the model just caches it in Setup. The **driver** builds the per-node 6-component field: - **TPV27 (now)**: sample the **same Phase-1 Tpv2627DepthStressSource** at each bulk node (the §1.2 depth formulas, code frame §2.4: spec total tensor T_{ij} tension-positive + $P_f = \text{water_density} \cdot g \cdot \max(0, -z)$ on the diagonal). Reusing the Phase-1 source guarantees fault and bulk σ^0 are consistent by construction. - **Full cycle (later, §4.4.8)**: $C:\epsilon(u_QD)$ from the interseismic solve (the existing QD $\rightarrow$ DYN handoff).

Rebuilt by the driver on restart (not checkpointed); $\text{water_density}/g$ come from the [stress] block. Decoupling the model from the σ^0 source is what lets the same DP core serve both the analytic-profile (TPV27) and QD-derived (cycle) cases without change.

1.5.4.6 4.4.6 Internal-state output (optional) eta (and optionally ep_ij) → ParaView via StateFieldNames() + the registration/copy pattern in §4.4.2. **Not a v13 spec-required output** — diagnostic only. (SeisSol stores all 7: ep_xx..ep_xz, eta, Model/Plasticity.h:80–81, for reference.)

1.5.4.7 4.4.7 Forward-compatibility — continuum damage–breakage model (CDBM) (*Design target only; NOT implemented here. Goal: confirm the §4.4.2 framework is the right seat for the separate CDBM port — see the companion plan in worktree cdbm-damage-breakage-plan / memory project_cdbm_mfem_port_plan, porting the MOOSE-FARMS continuum damage–breakage model and reusing the TPV26/27 fault half + the operator-split bulk update at the Q.Swap chokepoint.*)

CDBM (Lyakhovsky–Ben-Zion) evolves per-node internal scalars — damage $\alpha \in [0, 1]$ and breakage $B \in [0, 1]$ (plus optionally a granular/plastic strain) — and makes the elastic moduli $\lambda(\alpha)$, $\mu(\alpha)$ (and a damage-coupling modulus $\gamma(\alpha)$) **state-dependent**. It maps onto the framework, with **two net-new pieces** beyond what Drucker–Prager needs: - **Fits the interface as-is:** NumStateVars() ≥ 2 (α , B , +granular) via the generic rheo_state storage + checkpoint; Setup precomputes $\alpha_0(x)$ and $\varepsilon_0 = S(\alpha_0):\sigma^0$ from the same node_xyz/ σ^0 plumbing; per node ApplyStep reconstructs $\varepsilon = S(\alpha):\sigma$ (compliance inversion — pointwise, no gradients), forms invariants $I1$, $I2$, $\xi = I1/\sqrt{I2}$, integrates the α/B ODEs over dt (internal sub-stepping for stiffness), recomputes σ , writes back to Q. Same cadence/chokepoint/model-registry as DP - **NET-NEW (1) Damage-dependent moduli vs frozen flux Jacobians** (the EffectiveModuli() hook): the elastic wave step must see $\lambda(\alpha)$, $\mu(\alpha)$, but the wave operator bakes constant homogeneous Jacobians. CDBM routes EffectiveModuli() into the **per-element/heterogeneous flux path** (bimaterial_wave_operator.* / heterogeneous_material.* exist, §2) instead of the constant-Jacobian path. The interface **reserves** the hook now (returns nullptr for elastic/DP) so the loop structure need not change later. - **NET-NEW (2) Nonlocal Bažant-3D ξ -average across ranks** — *the one place the pointwise/no-MPI property of the DP path breaks*. CDBM regularizes localization by replacing the local ξ with a Bažant nonlocal-integral average over a neighborhood, which spans elements and **MPI ranks** $\Rightarrow$ a halo/ghost exchange. The framework accommodates this by extending BulkRheology::Context with the ParFiniteElementSpace/mesh + a nonlocal-average helper, and running a **nonlocal ξ pre-pass** inside ApplyStep before the pointwise damage update. These are kept out of the v1 (elastic/DP) interface deliberately (YAGNI) but the operator-split structure, the chokepoint, and the registry are unchanged — only Context grows and one collective is added. - Out of scope now: the CDBM constitutive math, the moduli→flux feedback, the nonlocal averaging + its MPI exchange, and CDBM config/tests. This plan delivers the framework + LinearElastic + DruckerPrager, leaving CDBM a *new BulkRheology file* + the two net-new pieces above (tracked by its own plan).

1.5.4.8 4.4.8 Full-cycle / quasi-dynamic-sharing seam (design target; only the dynamic adapter is built here) (*Per user direction: off-fault plasticity/viscoplasticity is wanted in the quasi-dynamic (interseismic) path too — distributed off-fault plastic strain accumulated over many cycles, slow viscoplastic bulk relaxation — not only coseismically. This subsection records how the Phase-5 design keeps that reachable without a later rewrite. No QD code is written in this deliverable.*)

The codebase already reserves the QD seat. The QD ConstitutiveModel ($\varepsilon \rightarrow \sigma$) is tiered: it de-

clares NumInternalVars(), HasStateEvolution(), ComputeStateRates(), NumNonLocalVars() with no-op defaults (constitutive/constitutive_model.hpp), and only LinearElastic implements it today. Those virtuals are the intended host for QD inelasticity; there is simply no member behind them yet.

Shared core, two adapters (not one unified interface). The two phases use genuinely different numerics — QD is implicit, strain-based, and needs a consistent algorithmic tangent; DYN is explicit, operator-split on Q. Forcing one interface to serve both makes each side carry baggage. Instead: | Layer | Home | Owns | |—|—|—| | **Constitutive core** (stateless) | constitutive/drucker_prager_core.{hpp,cpp} | yield fn, Duvaut–Lions return map, state-rate, σ^0 assembly — tensor-in/out | | **Dynamic adapter** (built now) | constitutive/drucker_prager_rheology.{hpp,cpp} (BulkRheology) | operator-split cadence on Q; per-node loop calling the core | | **QD adapter** (deferred, rewrite-free) | future ConstitutiveModel member | $\varepsilon \rightarrow \sigma$ embedding + the consistent tangent (the one QD-only addition) |

Two decisions taken now to keep the seam open (both cheap, both already in §4.4.2/§4.4.5):

1. **Injected σ^0** — ctx.sigma0_eff is supplied by the driver, so the same model runs on the analytic depth profile (TPV27) or $C:\varepsilon(u_QD)$ (cycle) unchanged. 2. **Cycle-persistent inelastic state** — eta/ep (and later α/B) are one continuous per-node field that both phases read and evolve; the generic rheo_state checkpoint (§4.4.2) already persists it across a QD phase. It must be owned at cycle scope, not as dynamic-run scratch.

Why plasticity vs damage still differ across the cycle. DP plasticity does not change moduli, so a QD DP model only needs the core + the tangent (the QD elastic stiffness is unchanged). **CDBM is the deeper case:** $\lambda(\alpha)$, $\mu(\alpha)$ are state-dependent, so the QD DG stiffness assembly *must* see the damaged moduli (interseismic loading through pristine stiffness would be physically inconsistent) — the same EffectiveModuli map (§4.4.7) that feeds the dynamic flux Jacobians must also feed the QD assembly. That coupling, and any QD adapter, are tracked by the full-cycle / CDBM work — not built here.

4.5 Mesh (TPV26 + TPV27 share one mesh)

Reparametrize tpv31/mesh/tpv31_50m.geo (cleanest base: canonical frame, surface-breaking embedding, minimal size field) $\rightarrow$ tpv26/mesh/tpv26_{200,100}m.geo (and a tpv26_50m.geo for convergence): - Fault rectangle on the $y=0$ plane: $x \in [-20000, 20000]$, $z \in [-20000, 0]$ (surface-breaking). - **Embed the fault top edge in the free surface:** Curve{<fault-top-edge>} In Surface{<free-surface>} (mandatory; otherwise “No elements in volume”). - Box half-extents from $d \geq c_p \cdot t_{final}/2 = 6000 \cdot 13/2 = 39$ km $\Rightarrow$ use $\sim \pm 50$ –60 km horizontal and ~ 50 km deep (absorbing walls, not true PML). - Size field (reuse verbatim, drop tpv205’s nucleation/stress-patch fields): Distance(fault) $\rightarrow$ MathEval “0.3*F1 + (F1/2.5e3)^2 + lc_fault” $\rightarrow$ Threshold(pin lc_fault then release to lc_far) $\rightarrow$ Min; Characteristic Length{PointsOf{ Surface{fault} } } = lc_fault. lc_fault = 100 (prod), 200 (smoke), 50 (convergence). - Algorithm3D=1 (Delaunay); **do NOT** enable Mesh.OptimizeNetgen (SIGABRT on surface-reaching faults); end with Mesh.MshFileVersion = 2.2;. - Build offline (Frontera): gmesh -format msh22 -3 tpv26_100m.geo -o tpv26_100m.msh (.msh is gitignored). - For TPV27, optionally widen the near-fault fine band (raise Threshold DistMax / lower MathEval slope) so the off-fault plastic zone (a few km) is resolved — tuning, not a structural change. - Config

[fault_geometry] ref_normal=[0,-1,0], up=[0,0,1], kind="vertical_strike_slip" (already correct); [boundary] attribute IDs must match the .geo Physical Surface tags (reuse tpv205's 103/101/105 to reuse existing config blocks, or tpv31's 101/102/103/104 — either works since they are config-driven).

4.6 Output

1.5.6.1 4.6.1 On-fault stations — dynamic/tpv2627_stations.hpp Clone tpv205_stations.hpp (9-col SCEC TPV5 layout) → 12 TPV26/27 stations, **8-col** SCEC layout (t, h-slip, h-slip-rate, h-shear-stress, v-slip, v-slip-rate, v-shear-stress, n-stress). - h (strike) = DOFData channel 2 (slip2/V2/tau2_corr); v (dip) = channel 1 (slip1/V1/tau1_corr), + = downward (can_t1=(0,0,-1)); n-stress = effective normal σ_{n_corr} — Pf (compression→sign per spec: report + = **extension**, so emit $-(\sigma_{n_eff})$ if σ_{n_corr} is compression-positive — pin by comparing one trace against the TPV205 writer's sign and the spec's "+ = extension"). - Emit the verbatim SCEC header tokens (§1.6) with element_size, time_step, num_time_steps, per-station location. Stresses in MPa, time format 20.12E, data 14.6E.

1.5.6.2 4.6.2 Off-fault surface stations — dynamic/tpv2627_surface_stations.hpp Reuse the TPV205SurfaceStationWriter structure + the **tpv6_stations.hpp displacement accumulator** (displacement = $\int$ velocity dt; note the documented restart-from-0 semantics on resume). 6 body stations, **7-col** (t, h-disp, h-vel, v-disp, v-vel, n-disp, n-vel). Component map (spec → code velocity): h-vel = VX (strike, +x); v-vel = -VZ (down = +depth = -z_code); n-vel = VY (fault-normal, +z spec $\approx$ code-y — verify sign against the §2.4 normal orientation); displacements are the time-integrals. *(The dynamic operator has no displacement state; the accumulator is the only displacement source — this is the main off-fault new capability.)*

1.5.6.3 4.6.3 Rupture-time cplot — new tracker No rupture-time tracking exists (grep-confirmed). Add a per-fault-DOF array rup_time initialized to 1e9; in the time loop, after the fault state update, set rup_time[i] = min(rup_time[i], t) the first time dof_data[i].slip_rate > 0.001. At run end, MPI-gather and write cplot with j = x, k = depth = -z, t = rup_time (reduced header per §1.6).

1.5.6.4 4.6.4 Tag dispatch drivers/spatial_dyn_driver.cpp:3294-3299: add else if (tag == "tpv26" || tag == "tpv27") { wire_stations((Tpv2627StationWriter*)nullptr, DefaultStations_Tpv2627()); } and wire the surface-station + cplot writers analogously.

4.7 Config TOMLs

tpv26/configs/tpv26_spatial_p1_100m.toml (+ _200m smoke, _50m, p2 variants), and tpv27/configs/tpv27_spa etc. (identical except [problem] tag and [rheology]). Skeleton from tpv205/configs/tpv205_spatial.toml; key blocks:

```
[meta]    schema_version=1; law="slip_weakening"; description="SCEC TPV26 (elastic) via spatial_dy
[problem] tag="tpv26"                # "tpv27" for the plastic case
[material_constant_fallback] lambda=3.2044e10; mu=3.2038e10; rho=2670.0
[pore_pressure] P_p_pa=0.0; P_p_grad_pa_per_m=9800.0; min_sigma_n_pa=0.0
[mesh]    path="tpv26/mesh/tpv26_100m.msh"; order=1
```

```
[velocity] use_sidecar=false
[stress] kind="tpv2627_depth"; rho=2670.0; g=9.8; b11=0.926793; b33=1.073206; b13=-0.169029
        omega_top_m=15000.0; omega_bot_m=20000.0; water_density=1000.0
[friction.slip_weakening] mu_s=0.18; mu_d=0.12; d_c=0.30
        cohesion_floor_pa=0.40e6; cohesion_grad_pa_per_m=720.0; cohesion_ref_depth_m=5000.0; coh
[nucleation] kind="forced_rupture"
[nucleation.forced_rupture] hypocenter_x=-5000.0; hypocenter_y=0.0; hypocenter_z=-10000.0
        rcrit_m=4000.0; vs=3464.0; vr_factor=0.7; t0_s=0.5
[rheology] kind="linear_elastic" # TPV27: kind="drucker_prager" + [rheology.drucker_prager]{cohesi
[numerics] ader_order=3; mixed_flux="adjacent"; cfl=0.25; cfl_safety="dg"; interior_flux="scalar";
[time] tfinal="13.0s"; t_initial=0.0; dt_initial="auto"
[boundary] fault_attr=103; natural_attrs=[101]; absorbing_attrs=[105]
[fault_geometry] ref_normal=[0.0,-1.0,0.0]; up=[0.0,0.0,1.0]; kind="vertical_strike_slip"
[output] output_dir="..."; (uniform resample cadence ~0.005-0.008 s)
```

4.8 Build wiring (Makefile)

- New .cpp: constitutive/drucker_prager_rheology.cpp and constitutive/bulk_rheology_factory.cpp (link into seas_spatial_dyn_driver); bulk_rheology.hpp and linear_elastic_rheology.hpp are header-only. The new stress source goes in spatial/code/spatial_stress.cpp and ResolveForcedRupture in spatial/code/spatial_friction.cpp (both already built objects).
- New station headers are header-only (no .o).
- New test targets (§5/§6): seas_test_tpv2627_stress_source, seas_test_bulk_rheology_framework, seas_test_drucker_prager_rheology, seas_test_forced_rupture_resolver (revive #if 0), seas_test_forced_rupture_iterator_parity, seas_test_tpv2627_cohesion_taper, seas_test_linear_elastic_rheology_byte_identical.

1.6 5. Phased implementation plan

Each phase is independently testable and leaves the tree compiling + make test green. Phases 0–4 are TPV26-shared; phases 5a–5b add the nonlinear path (5b is TPV27-only). Read the **In one sentence** lines top-to-bottom for the arc; drop into a phase’s Detailed requirements for the contract.

[CORR] Rev-4 code-verified corrections (2026-07-08). Three earlier-rev assumptions do not hold in the current worktree and are fixed below (details at each use site): 1. **Nucleation resolution is free-function-based, not a resolver method.** SpatialFrictionResolver has no nucleation entry point; the 3 existing kinds are resolved by free functions in dynamic/spatial_nucleation.hpp (ResolveGradualOverstress :145, ...CompactCircular :237, ...InstantaneousOverstressCircular :316). ⇒ ResolveForcedRupture is a **new free function in dynamic/spatial_nucleation.{hpp,cpp}** (Phase 2), *not* SpatialFrictionResolver::ResolveForcedRupture. 2. **NucleationKind currently has exactly 3 members** (GradualOverstress, GradualOverstressCompactCircular, InstantaneousOverstressCircular; spatial_friction.hpp:437). The old StrengthReduction/Overstress kinds were removed. ForcedRupture is a *new* enum member. 3. **The archived #if 0 forced-rupture tests must be adapted, not revived verbatim** — they call the removed

SpatialFrictionResolver::ResolveForcedRupture(...) and read NucleationSpec fields (hypocenter_x_m, r_crit_m, t0_decay_s) that no longer exist. The $T(r)$ formula and golden values (test file lines 904–1203) are reusable; the call shape and config fields are not.

Verified-current anchors used throughout §5 (driver = drivers/spatial_dyn_driver.cpp, **3723 lines**): chokepoint Q.Swap(Q_new) :3364 then t += dt_step :3365; stress-seed arms :1683–1806; SetFaultFrictionLaw :1397; dummy T_forced=1e9/t0=0 + InitializeFaultDOFs_Spatial :1974–2002; checkpoint fns are Read/WriteTpv104Checkpoint (read :2940, writes :3648/:3692); ParaView bulk registration :2730–2765, per-cycle copy :3157–3175; station-writer tag dispatch :3269–3300 (per-step stations_write(t, dof_data) :3565). Wave-operator accessors: GetScalarNDof() :203, GetFESpace() (const) :194, GetMesh() :776; **no node-coordinate accessor exists** — a bulk node_xyz table must be built (mirror BuildPerDOFFaultTables, driver :270).

5.0 Contract format + unit-test harness conventions (read once)

Each phase is a self-contained contract: **In one sentence** · **Goal** · **Files to create/modify (with anchors)** · **Detailed requirements (exact signatures)** · **Edge cases** · **Unit tests** · **Acceptance criteria** · **Dependencies**. Math is stated in §4 and referenced, not repeated.

Unit-test harness — applies to every new tests/unit/test_*.cpp below (verified against the **current tree**): - **No test framework** (no Catch2/GoogleTest). A test is a plain executable: include mfem.hpp, the headers under test via .././../, then #include "test_macros.hpp" (shared macros TEST_ASSERT(cond,msg), TEST_NEAR(val,exp,tol,msg), TEST_REL_NEAR(val,exp,rtol,msg), TEST_PRINT_RESULTS(); tests/unit/test_macros.hpp:24–83). - int main() calls each void Test...() then TEST_PRINT_RESULTS(); return num_failed == 0 ? 0 : 1; **The exit code is the pass/fail contract** the make test targets depend on. - Prefer **standalone** (no MPI_Init, no mesh file): build an inline Mesh::MakeCartesian3D(...) when a mesh is required (cf. test_constitutive_integrator.c). Use the **graceful-skip idiom** (print "...unavailable; compile-only"; return 0;; cf. test_compute_safs_params.c) when a fixture mesh is not present locally. - **Makefile** — **4 edits per new test** (mirror the constant_tensor_sign target verbatim): (a) TEST_X_SRC / TEST_X_OBJ vars (Makefile:169); (b) obj-compile static-pattern rule, adding \$(HDF5_INCFLAGS) \$(TOML_FLAGS) **only if** the test pulls in spatial-config/stress code (Makefile:2712); (c) seas_test_X: link rule + test-x: run target (Makefile:2720); (d) register the executable in SEQ_MINIAPPS (Makefile:902) **and** the test-x run target in the test: aggregate (Makefile:4007). **Link sets**: a header-only / inline-mesh test links only \$(MFEM_LIBS); a test that calls spatial::LoadSpatialFrictionConfig/ParseSpatialFrictionConfigString or constructs a stress source links \$(SPATIAL_FRICTION_OBJ) \$(SPATIAL_STRESS_OBJ) \$(STRESS_FIELD_3D_OBJ) \$(DATA_FIELD_3D_OBJ) \$(FIELD_COEFFICIENT_OBJ) \$(HETEROGENEOUS_MATERIAL_OBJ) \$(MATERIAL_COEFFICIENT) \$(MFEM_LIBS) \$(HDF5_LIBS).

Phase 0 – Scaffolding (no physics)

In one sentence: a tpv26/tpv27 config loads and runs a few steps through the *existing* pipeline (plain LSW + gradual overstress), producing nothing new but proving the plumbing.

Goal. Stand up the directory layout, a tiny mesh, and a [problem] tag that the driver recognizes,

reusing an existing station writer so end-to-end smoke works before any new physics lands.

Files to create. - `tpv26/configs/tpv26_spatial_smoke.toml`, `tpv27/configs/tpv27_spatial_smoke.toml` — from `safs/tpv205` skeletons; initially `[nucleation] kind="gradual overstress"`, `[stress] kind="constant_tensor"`, `[rheology] absent`, tag `"tpv205"` (so the existing writer wires) — physics-faithful blocks arrive in later phases. - `tpv26/mesh/tpv26_smoke.geo` — a coarse (~1–2 km) surface-breaking box, small enough to run locally in seconds. - Copy the spec PDF into `tpv27/benchmark_document/` (already present under `tpv26/`).

Files to modify. None yet (tag `"tpv205"` reuses the existing dispatch). The real `"tpv26"/"tpv27"` dispatch arm is added in Phase 4.

Detailed requirements. 1. Mesh emitted **Gmsh v2.2** (`gmsh -format msh22 -3 ...`) per CLAUDE.md's v2.2-only parser constraint. 2. The smoke config must pass `LoadSpatialFrictionConfig` (`spatial_friction.cpp:1784`) with no new keys.

Edge cases. Local run must respect the memory rule *no production-mesh local runs* — the smoke mesh is a tiny fixture only.

Unit tests. None (integration smoke only).

Acceptance criteria. - `[ ] seas_spatial_dyn_driver --config tpv26/configs/tpv26_spatial_smoke.toml` loads and advances ≥ 5 steps with no abort, on the smoke mesh, $np \leq 4$ locally. - `[ ] make test` still green (nothing touched).

Dependencies. Depends on: nothing. Required by: all later phases (provides the config/mesh skeleton).

Phase 1 – Depth-dependent initial stress source (TPV26 + TPV27)

In one sentence: the fault sees the SCEC depth-dependent prestress, so the on-fault shear/normal ratio at 10 km depth equals the spec's 0.1575.

Goal. Add `Tpv2627DepthStressSource` (the §1.2 depth profile) as a new `StressSource3D`, wired through the kind enum, TOML parser, validator, and the driver's stress-seed dispatch, reusing `FaultGeometry::ComputeParams` verbatim.

Files to create. (*extend existing files, no new file*) - `spatial/code/spatial_stress.{hpp,cpp}` — add class `Tpv2627DepthStressSource`.

Files to modify. - `spatial/code/spatial_stress.hpp` — declare the class after `DepthProportionalToShearModulus` (:132–164, the model to copy). - `spatial/code/spatial_stress.cpp` — define ctor + `Evaluate` after :151. - `spatial/code/spatial_friction.hpp` — `StressSourceKind::Tpv2627Depth` (append to enum :224–230); new struct `Tpv2627DepthStressSpec` (mirror `DepthProportionalStressSpec` :236–245); add field to `StressSpec` (:278–304). - `spatial/code/spatial_friction.cpp` — extend `parse_stress_kind` (:423, add `"tpv2627_depth"`); add a validation branch in `parse_root's` `[stress]` block (:941–1092). - `drivers/spatial_dyn_driver.cpp` — add an else if (`cfg.stress.kind == StressSourceKind::Tpv2627Depth`) arm in the stress-seed chain (:1683–1806) that builds the source and calls `geom.ComputeParams(src, P_p_pa, P_p_grad_pa_per_m, min_sigma_n_pa)`.

Detailed requirements. 1. The class satisfies the duck-typed `StressSource3D` concept (no virtual

base): cpp class Tpv2627DepthStressSource { public: Tpv2627DepthStressSource(real_t rho, real_t g, real_t water_density, real_t b11, real_t b33, real_t b13, real_t omega_top_m, real_t omega_bot_m); mfem::DenseMatrix Evaluate(real_t x, real_t y, real_t z) const; // 3x3 sym, compression-POSITIVE const std::array<real_t,6>& BBox() const { return bbox_; } // infinite box bool ContainsBBox(real_t,real_t,real_t,real_t,real_t,real_t,real_t=0.0) const { return true; } private: real_t rho_,g_,wd_,b11_,b33_,b13_,omega_top_,omega_bot_; std::array<real_t,6> bbox_; }; 2. Evaluate computes depth = max(0,-z), then Pf, σ_{22} , Ω , σ_{11} , σ_{33} , σ_{13} per §1.2, assembles the tensor in the **code frame** via the §2.4 remap ($\sigma_{xx}=\sigma_{11}$, $\sigma_{yy}=\sigma_{33}$, $\sigma_{zz}=\sigma_{22}$, $\sigma_{xy}=\sigma_{13}$), and returns it in **compression-positive** convention (negate the tension-positive spec diagonal). ctor validates rho>0, g>0, omega_bot>omega_top_ with MFEM_VERIFY (LinearElastic style); bbox_ = all $\pm\text{inf}$. 3. **[CORR — sign, load-bearing]** Unlike ConstantTensor/DepthProportional, the shear here is computed *inside* Evaluate (spec σ_{13}), so the driver arm does **not** pass a -cfg.stress.sigma_xy_pa to negate (that construction-time negation, driver :1702/:1794, only applies to a config-supplied sigma_xy). Therefore the right-lateral-positive sign must be baked into the σ_{xy} component returned by Evaluate so that after ComputeParams' plain projection $\tau_{\text{strike}} = t2 \cdot (S \cdot n)$ (no external flip; fault_geometry_safs_templated.inl:83-127) the hypocenter $\tau_{\text{strike}} = +27.66$ MPa. **Validate by golden test — do not assume the sign.** 4. Config knobs (defaults = spec): rho=2670, g=9.8, water_density=1000, b11=0.926793, b33=1.073206, b13=-0.169029, omega_top_m=15000, omega_bot_m=20000. Pore pressure via the existing [pore_pressure] P_p_grad_pa_per_m=9800 (so ComputeParams yields effective $\sigma_n = Pf$).

Interfaces. Tpv2627DepthStressSource::Evaluate(x,y,z); StressSourceKind::Tpv2627Depth; the parser string "tpv2627_depth".

Edge cases. - $z = 0$ (surface node): depth=0 $\Rightarrow$ Pf=0, $\sigma_{22}=0$; Evaluate returns the zero-ish tensor; ComputeParams pore-pressure subtraction gives Pf=0 exactly (§4.1 open item; near-surface C0=4 MPa dominates). - depth $\in [15000, 20000]$: Ω linear taper; depth ≥ 20000 : $\Omega=0$ ($\sigma_{11}=\sigma_{33}=\sigma_{22}$, shear $\rightarrow 0$). - depth < 0 guarded to 0 by max(0,-z) inside both Evaluate and ComputeParams.

Unit tests — tests/unit/test_tpv2627_stress_source.cpp (Style A; links the spatial set + \$(MFEM_LIBS) \$(HDF5_LIBS); standalone, no mesh — call Evaluate directly and hand-project onto the canonical fault basis $n=(0,-1,0)$, $t1=(0,0,-1)$, $t2=(1,0,0)$): - Test_Hypocenter_Ratio() — at (x=-5000,y=0,z=-10000) (10 km depth, $\Omega=1$): TEST_NEAR(sigma_n_eff, 175.64e6, 1e3), TEST_NEAR(tau_strike, 27.66e6, 1e3), TEST_NEAR(tau_strike/sigma_n_eff, 0.1575, 1e-3), and TEST_ASSERT(0.12 < ratio && ratio < 0.18, ...) (between μ_d and μ_s). - Test_RightLateral_Sign() — TEST_ASSERT(tau_strike > 0, "right-lateral positive") (pins requirement 3). - Test_Omega_Taper() — depths 15000/17500/20000 km give $\Omega = 1 / 0.5 / 0$; verify $\sigma_{11}, \sigma_{33} \rightarrow \sigma_{22}$ as $\Omega \rightarrow 0$. - Test_Surface_ZeroPf() — z=0 $\Rightarrow$ Pf=0, tensor finite, no NaN. - Test_Dip_Shear_Zero() — tau_dip ≈ 0 ($\sigma_{12}=\sigma_{23}=0 \Rightarrow$ no down-dip prestress on a pure strike-slip fault).

Acceptance criteria. - [] All five test_tpv2627_stress_source cases pass. - [] A tpv26 config with [stress] kind="tpv2627_depth" loads, seeds the fault, geom.HasParams() true. - [] make test green; no other stress kind's behavior changes (new enum arm is additive).

Dependencies. Depends on: Phase 0. Required by: Phase 4 (outputs need a seeded fault), Phase 5b (σ^0_{eff} reuses the same depth formulas).

Phase 2 – Forced-rupture nucleation, end-to-end (TPV26 + TPV27)

In one sentence: rupture is *forced* to initiate at the hypocenter and spread per the spec's $T(r)$ front — on **every** fault node (interior included), not just MPI seam faces — without adding any near-fault shear stress.

Goal. Add the forced-rupture time-weakening path: a per-DOF $T(r)$ resolver, a new nucleation kind + parser, driver wiring to feed real T_{forced}/t_0 into the fault DOFs, and the **round-6 fix** so the interior friction iterator applies the time-dependent $\mu(\delta, t)$.

Files to create. - `dynamic/spatial_nucleation.{hpp,cpp}` — **[CORR]** add free function `ResolveForcedRupture + ForcedRupturePerDOFParams` (alongside the existing 3 resolvers), *not* a `SpatialFrictionResolver` method.

Files to modify. - `spatial/code/spatial_friction.hpp` — `NucleationKind::ForcedRupture` (append to the 3-member enum :437); new struct `ForcedRuptureSpec { real_t hypocenter_x,y,z; real_t rcrit_m; real_t vs; real_t vr_factor; real_t t0_s; }` as a member of `NucleationSpec` (:454–461). - `spatial/code/spatial_friction.cpp` — add a `kind == "forced_rupture"` branch in the `[nucleation]` parser (:1321–1414) reading `[nucleation.forced_rupture]`; extend the trailing `MFEM_ABORT` valid-kinds list. - `dynamic/friction_substep_iterator.{hpp,cpp}` — **round-6 fix:** thread the substep absolute time into the interior LSW step. - `dynamic/friction_iterator_factory.cpp` — pass the forced-rupture flag when constructing the LSW iterator. - `dynamic/wave_operator.hpp` — add `FaultFrictionLaw::LSW_ForcedRupture` (currently only LSW/RateAndState). - `drivers/spatial_dyn_drive` — `SetFaultFrictionLaw` (:1397) selects `LSW_ForcedRupture` when the nucleation kind is forced rupture; replace `dummy_T_forced=1e9/dummy_t0_decay=0` (:1974–1975) with the resolver output passed to `InitializeFaultDOFs_Spatial` (:1982).

Detailed requirements. 1. Resolver (new free function, signature mirrors `ResolveGradualOverstress`):
`cpp struct ForcedRupturePerDOFParams { mfem::Vector T_forced_s, t0_decay_s; };`
`// size = num_fault_dofs ForcedRupturePerDOFParams ResolveForcedRupture(const`
`spatial::ForcedRuptureSpec& spec, const`
`mfem::Vector& dof_coords_3d /*3*N*/);` For each fault DOF at code $(x, \cdot, z)$: $r = \sqrt{(x-x_{\text{hyp}})^2 + (z-z_{\text{hyp}})^2}$; if $r < r_{\text{crit}}$, $T = r/(v_r \cdot v_s) + 0.081 \cdot r_{\text{crit}}/(v_r \cdot v_s) \cdot (1/(1-(r/r_{\text{crit}})^2) - 1)$, else $T = 1e9$; $t_0_decay_s = t_0_s$ uniformly. **Formula is copied verbatim from the archived test** (see Unit tests). 2. **Round-6 fix (byte-exact-sensitive; dynamic/friction_substep_iterator.cpp).** Today `LinearSlipWeakeningIterator::StepOneQP_` (:197–250) calls `LSWFrictionCoefficient_TPV205(delta,...)` with **no time term** (:221–224). The $t_{\text{sub_end}}$ absolute time is already computed at the call-site `lambda` (:314–321, `friction_substep_iterator.hpp`:210–216) but discarded. Change: - Add a `real_t t_sub_end` parameter to `StepOneQP_` (decl `friction_substep_iterator.hpp`:352–359) and forward it from the `lambda` (:321). - Add a `bool forced_rupture_member` to `LinearSlipWeakeningIterator`, set by the factory (req. 3). - When `forced_rupture` is true, call `spatial::LSWFrictionCoefficient_ForcedRupture` (`d.lsw_mu_s, d.lsw_mu_d, d.lsw_d_c, t_sub_end, d.T_forced_rupture, d.t0_decay_forced`) (`spatial_friction.hpp`:813); otherwise the **literally unchanged** `LSWFrictionCoefficient_TPV205(...)` call. - **Byte-exact contract:** the plain-LSW branch is textually identical to today; the forced-rupture

helper reduces byte-identically to plain LSW at $T_{\text{forced}} \geq 1e8$ (already proven by the live `H_1_T_1e9_reduces_to_plain_LSW` test, `test_spatial_friction_resolver.cpp:1298–1314`, `tol 0.0`). - Interior (iterator) and seam (`fault_face_flux.cpp:1088–1094`, which already calls the same helper) must use identical $\mu(\delta, t)$ to avoid an MPI seam discontinuity. 3. `MakeFrictionIterator` (`friction_iterator_factory.cpp:26–108`): in the `SlipWeakening` case (`:28–32`), construct `std::make_unique<LinearSlipWeakeningIterator>(flux, /*forced_rupture=*/ cfg.nucleation.kind == NucleationKind::ForcedRupture)` (add the ctor arg, default `false` to keep other callers unchanged). 4. Forced rupture is a **friction-weakening** mechanism — it produces **no** `tau_nuc` stress increment (unlike the overstress kinds). The `nuc_cb` stress-accumulator path is untouched; forced rupture flows purely through the per-DOF `T_forced_rupture/t0_decay_forced` fields already carried in `DOFData` (`fault_face_flux.hpp:113–140`) and the μ computation.

Interfaces. `ResolveForcedRupture(spec, dof_coords_3d) → ForcedRupturePerDOFParams; NucleationKind::ForcedRupture; FaultFrictionLaw::LSW_ForcedRupture; the new LinearSlipWeakeningIterator(bool forced_rupture) ctor.`

Edge cases. - $r \geq r_{\text{crit}} \Rightarrow T=1e9 \Rightarrow$ node never forced (byte-identical to plain LSW there). - $r = 0$ (a DOF exactly at the hypocenter) $\Rightarrow T(0)=0$ (forced from $t=0$). - $t0_decay = 0 \Rightarrow$ step ramp; guard division in the helper (it already handles $t0=0$). - Restart: `T_forced_rupture/t0_decay_forced` are per-DOF `DOFData` fields already threaded by `InitializeFaultDOFs_Spatial`; no checkpoint change.

Unit tests. - `tests/unit/test_forced_rupture_resolver.cpp` (Style A; links spatial set; standalone) — **adapt** the archived `#if 0` block `test_spatial_friction_resolver.cpp:904–1203`: - Reuse the $T(r)$ expression verbatim (`:941–944`): $\text{denom} = 1 - (r/r_{\text{crit}})^2$; $T = r/(0.7 \cdot V_s) + 0.081 \cdot r_{\text{crit}}/(0.7 \cdot V_s) \cdot (1/\text{denom} - 1)$. - `Test_T_of_r()` — DOFs at $r \in \{0, 1000, 2000, 3000, 4500\}$ m, $r_{\text{crit}}=4000$, $V_s=3464$, $vr_factor=0.7$, $t0=0.5$: `TEST_NEAR(T_forced_s(i), T_expect, 1e-6)`; `TEST_ASSERT(T_forced_s(4)==1e9)` ($r=4500 > r_{\text{crit}}$); all `t0_decay_s(i)==0.5`; $T(0)=0$; strictly monotonic increasing on $[0, r_{\text{crit}})$. - **[CORR]** call the *new free function* `ResolveForcedRupture(spec, coords)`, not the removed `R.ResolveForcedRupture(nuc, dofs, elem, mat, mesh)`; populate a `ForcedRuptureSpec`, not the deleted `NucleationSpec::hypocenter_x_m/r_crit_m/t0_decay_s` fields. - `tests/unit/test_forced_rupture_iterator_parity.cpp` (Style A; links the fault/iterator objects + `$(MFEM_LIBS)`) — the round-6 byte-exact guard: - `Test_ForcedRupture_Reduces_To_LSW()` — for $\delta \in \{0, 0.1 \cdot d_c, d_c, 2 \cdot d_c\}$, assert `LSWFrictionCoefficient_ForcedRupture( $\delta$ ,  $\mu_s$ ,  $\mu_d$ ,  $d_c$ ,  $t=5.0$ ,  $T_{\text{forced}}=1e9$ ,  $t0=0.0$ ) == LSWFrictionCoefficient_TPV205( $\delta$ ,  $\mu_s$ ,  $\mu_d$ ,  $d_c$ )` at `TEST_NEAR(..., 0.0)` (bit-exact; extends the existing `H_1` check into a dedicated target). - `Test_ForcedRupture_Front()` — a small fixture where $t_{\text{sub_end}}$ crosses T_{forced} for one DOF: μ drops from μ_s toward μ_d at the right time; a $T_{\text{forced}}=1e9$ DOF stays at plain-LSW μ .

Acceptance criteria. - [] `test_forced_rupture_resolver` matches the SCEC $T(r)$ values (adapted from `:904–1203`). - [] `test_forced_rupture_iterator_parity` shows the interior path is byte-identical to TPV205 at $T=1e9$. - [] TPV205 native + spatial regression traces unchanged (round-6 change is gated on `forced_rupture`). - [] A tiny smoke: rupture initiates at the hypocenter and the front honors $T(r)$ on **interior** faces (not just seams); MPI seam/interior $\mu(\delta, t)$ consistent on $np \geq 2$.

Dependencies. Depends on: Phase 0. Required by: Phase 4 (a rupture must exist to output).

Phase 3 – Depth-dependent frictional cohesion (TPV26 + TPV27, config-only)

In one sentence: near-surface fault strength is raised by the spec's cohesion taper (4 MPa at the surface → 0.4 MPa below 5 km) using existing resolver code — zero new C++.

Goal. Set the [friction.slip_weakening] cohesion taper so `SpatialFrictionResolver::ResolveSlipWeakening` (`spatial_friction.cpp:1833`) produces the spec $C_0(\text{depth})$.

Files to create. None. **Files to modify.** The `tpv26/tpv27` config TOMLs only.

Detailed requirements. 1. In [friction.slip_weakening]: `cohesion_floor_pa=0.40e6`, `cohesion_grad_pa_per_m=720.0`, `cohesion_ref_depth_m=5000.0`, `cohesion_taper_axis="z"` (`depth = max(0, -z)`). The resolver's $C_0 = \max(\text{floor}, \text{floor} + \text{grad} \cdot (\text{ref_depth} - \text{depth}))$ then yields 4.0 MPa at surface, 0.40 MPa at/below 5 km.

Edge cases. $\text{depth} > 5000 \Rightarrow \text{grad} \cdot (\text{ref_depth} - \text{depth}) < 0 \Rightarrow \max(\text{floor}, \dots)$ clamps to 0.40 MPa (verify the resolver's clamp direction with the test below).

Unit tests — `tests/unit/test_tpv2627_cohesion_taper.cpp` (Style A; links spatial set; standalone — parse a small inline TOML via `ParseSpatialFrictionConfigString` (`spatial_friction.cpp:1807`), call `ResolveSlipWeakening` on a hand-built few-DOF coord vector): - `Test_Cohesion_Taper()` — depths 0 / 2500 / 5000 / 10000 m $\Rightarrow$ TEST_NEAR(C_0 , {4.0e6, 2.2e6, 0.40e6, 0.40e6}, 1e3).

Acceptance criteria. - [] `test_tpv2627_cohesion_taper` passes (4.0 / 2.2 / 0.40 / 0.40 MPa). - [] make test green (config-only; no code path changes).

Dependencies. Depends on: Phase 1 (fault seeded). Required by: Phase 4.

Phase 4 – TPV26 outputs + first elastic run

In one sentence: TPV26 writes the three SCEC-format products — 12 on-fault stations, 6 off-fault surface stations (with displacement), and the rupture-time cplot — with correct headers, signs, and units.

Goal. Clone the existing writers into TPV26/27-specific ones, add the missing rupture-time tracker, and wire the "tpv26"/"tpv27" tag dispatch; run TPV26 elastic to first light.

Files to create. - `dynamic/tpv2627_stations.hpp` — on-fault 12-station, **8-column** writer (clone `tpv205_stations.hpp`, drop the trailing `mu_eff` 9th column). - `dynamic/tpv2627_surface_stations.hpp` — off-fault 6-station, **7-column** writer.

Files to modify. - `drivers/spatial_dyn_driver.cpp` — add `else if (tag == "tpv26" || tag == "tpv27")` to the station dispatch (:3269–3300); add the surface-station + cplot wiring; add rupture-time update in the per-step block (near :3565) and an end-of-run cplot writer.

Detailed requirements. 1. **On-fault writer** (`tpv2627_stations.hpp`): mirror `TPV205StationWriter` (`tpv205_stations.hpp:100`) — `Open(output_dir, prefix, stations, fault_coords, ndof[, comm])` (:104/:125, MPI MPI_MIN distance + rank tie-break), `WriteStep(t, dof_data)` (:189), `Close()`. Columns (`std::scientific`, time setprecision per §1.6): `t`, `slip2`, `v2`, `tau2_corr`, `slip1`, `v1`, `tau1_corr`, `n-stress` — i.e. **h = strike = channel 2**, **v = dip = channel 1**; drop the `mu_eff` column. `DefaultStations_Tpv2627()` = the 12 stations from §1.6. 2. **n-stress sign**

— spec reports effective normal with $+ = \text{extension}$. `dof_data.sigma_n_corr` is compression-positive; emit $-(\text{sigma_n_corr})$ (already effective via the `[pore\_pressure]` gradient). Pin the sign by comparing one trace against the TPV205 writer output in a unit test. 3. **Off-fault surface writer** (`tpv2627_surface_stations.hpp`): clone the **TPV205SurfaceStationWriter** structure — it lives in `dynamic/tpv205_setup.hpp:213` (not the lean `*_stations.hpp` header) — and borrow the **trapezoidal displacement accumulator** `disp += 0.5*(v_prev+v)*dt` idiom from `tpv6_stations.hpp:151–186` (`near_disp/vprev_members:261–264`). 6 body stations (§1.6), 7 columns `t`, `h-disp`, `h-vel`, `v-disp`, `v-vel`, `n-disp`, `n-vel`; velocity map `h=VX`, `v=-VZ`, `n=VY` (verify `n` sign against §2.4). **[CORR]** `tpv6_stations.hpp` is an *on-fault per-side* writer, not an off-fault surface writer — it is cited only as the $\int v \, dt$ idiom source; the surface-writer *structure* comes from `tpv205_setup.hpp:213`. 4. **cplot rupture-time tracker** (new; none exists): a per-fault-DOF `mfem::Vector rup_time(num_fault_total)` init `1e9`; in the per-step block (`:3565`) set `rup_time[i] = min(rup_time[i], t)` the first time `dof_data[i].V_abs > 0.001`. At run end MPI-gather and write cplot with `j = x`, `k = depth = -z`, `t = rup_time`, reduced header (§1.6). 5. **Output cadence** — resample on-fault/off-fault series onto a uniform grid ($\sim 0.005\text{--}0.008$ s) with `20.12E` time stamps, decoupled from the variable ADER/CFL `dt` (§1.6). 6. **SCEC header tokens** — emit the verbatim tokens (`# problem=`, `# author=`, ..., `# location=`) per §1.6; cplot uses the reduced header.

Edge cases. Restart truncates trace files $\Rightarrow$ displacement accumulator restarts from 0 (documented `tpv6_stations.hpp:28–31`); surface-breaking top node `Pf=0` (Phase 1 open item). A station with no owning rank (off-mesh) must be skipped, not crash (mirror TPV205’s ownership logic).

Unit tests. - `tests/unit/test_tpv2627_station_format.cpp` (Style A; header-only + `$(MFEM_LIBS)`; standalone) — `Test_OnFault_Columns()`: feed a synthetic `DOFData` with known `slip1/slip2/V*/tau*_corr/sigma`; assert the written row has 8 fields in the right order, MPa units, `n-stress = -sigma_n_corr` ($+ = \text{extension}$), and `h/v` map to channel 2/1. `Test_Header_Tokens()`: assert every required `#` token is present; cplot header omits `time_step/num_time_steps/location`. - `tests/unit/test_rupture_time_tracker.cpp` (Style A; standalone) — `Test_First_Crossing()`: drive a synthetic slip-rate history per DOF; assert `rup_time` records the **first** `t` with `V_abs>0.001` and stays `1e9` for never-rupturing DOFs. - `tests/unit/test_displacement_accumulator.cpp` (Style A; standalone) — `Test_Trapezoidal()`: a constant velocity `v` over `n` uniform steps gives `disp = v*t` to round-off; a linear ramp integrates exactly.

Acceptance criteria. - [] The three format/tracker/accumulator unit tests pass. - [] TPV26 on a coarse mesh produces well-formed on-fault / off-fault / cplot files with correct headers, signs, and units; on-fault `n-stress` is effective and “ $+ = \text{extension}$ ”. - [] **Frontera (user-approved)**: 100 m and 50 m, $0 \rightarrow 13$ s; compare on-fault slip/rate/stress at the 12 stations, off-fault `disp/vel` at the 6 body stations, and the rupture-time contour vs the SCEC TPV26 reference solutions.

Dependencies. Depends on: Phases 1–3. Required by: Phase 5b production comparison (same output path).

Phase 5a – Nonlinear bulk-rheology framework + linear-elastic no-op

In one sentence: a swappable `BulkRheology` interface is added and wired at the one integrator chokepoint, with a no-op `LinearElastic` default that keeps **every existing problem byte-identical**.

Goal. Build the operator-split constitutive path (§4.4): interface, no-op member, factory, [rheology] config, and driver wiring (bulk node_xyz, Setup, gated ApplyStep, checkpoint, ParaView state) — with a provable zero-overhead / byte-exact guarantee when the default is selected.

Files to create. - constitutive/bulk_rheology.hpp — the interface (§4.4.2), header-only, namespace mfem::seas, mirroring the constitutive_model.hpp style (pure-virtual base + defaulted forward-compat hooks). - constitutive/linear_elastic_rheology.hpp — the no-op member (§4.4.3), header-only. - constitutive/bulk_rheology_factory.{hpp,cpp} — MakeBulkRheology(const RheologyBlock&).

Files to modify. - spatial/code/spatial_friction.hpp — struct RheologyBlock { enum class Kind {LinearElastic, DruckerPrager} kind; DruckerPragerParams dp; }; add std::optional<RheologyBlock> rheology; to SpatialFrictionConfig (members :647–655). - spatial/code/spatial_friction.cpp — void parse_rheology(const toml::value&, RheologyBlock&) (mirror the inline [material] enum-kind block :1650–1670) + dispatch in parse_root (if (root.contains("rheology"))). - drivers/spatial_dyn_driver.cpp — (a) build a bulk node_xyz Vector(3*ndof_total) from GetFESpace() + GetMesh() (mirror BuildPerDOFFaultTables :270); (b) rheology = MakeBulkRheology(cfg.rheology); allocate Vector rheo_state(rheology->NumStateVars()*ndof rheology->Setup(ctx, rheo_state); (c) **hook before t += dt_step (:3365)**, after Q.Swap(Q_new) (:3364): if (rheology->NumStateVars() > 0) rheology->ApplyStep(ctx, dt_step, Q, rheo_state); — note Q holds the new state post-swap and t is still step-begin (t - dt_step semantics if the model needs begin-time); (d) checkpoint rheo_state alongside Q in Read/WriteTpv104Checkpoint (read :2940, writes :3648/:3692); (e) register StateFieldNames() as ParaView domain fields + per-cycle copy (mirror the bulk-stress block :2730–2765 / :3157–3175).

Detailed requirements. 1. Interface exactly as §4.4.2: NumStateVars(), Setup(ctx, state), ApplyStep(ctx, dt, Q, state)→size_t, StateFieldNames(), EffectiveModuli(ctx)→const Vector* (returns nullptr here; reserved for CDBM), Name(). Context { int ndof_total; const real_t* node_xyz; real_t lambda,mu,rho; }. 2. LinearElasticRheology: NumStateVars()==0; empty Setup/ApplyStep; EffectiveModuli→nullptr; Name()=="linear_elastic". **Byte-identity:** the driver gates ApplyStep on NumStateVars()>0, so the no-op path is never entered ⇒ TPV26 and all existing problems are provably unchanged. 3. MakeBulkRheology: LinearElastic default; unknown kind ⇒ MFEM_ABORT with a clear message. 4. [rheology] absent ⇒ cfg.rheology stays nullopt ⇒ driver treats as LinearElastic (default path).

Interfaces. BulkRheology (all methods §4.4.2); MakeBulkRheology(cfg.rheology); RheologyBlock.

Edge cases. [rheology] absent (default); NumStateVars()==0 skips allocation + hook + checkpoint; Q layout is component-major Q[c*ndof_total + dof] (confirmed by the ParaView copy :3161–3172) — the bulk node_xyz ordering must match the scalar-dof index so node_xyz[dof] aligns with Q[c*ndof_total+dof].

Unit tests — tests/unit/test_bulk_rheology_framework.cpp (Style A; links bulk_rheology_factory.o + spatial set + \$(MFEM_LIBS); standalone) using a local MockRheology (NumStateVars()==2, ApplyStep increments state deterministically): - Test_LinearElastic_NoOp() — NumStateVars()==0; ApplyStep leaves Q bit-identical (TEST_NEAR(...,0.0)). - Test_Factory_Default_And_Reject() — nullopt/"linear_elastic" → LinearElastic; unknown kind aborts (guarded — verify the mes-

sage, e.g. via a death-style check or a try wrapper if available). - `Test_State_RoundTrip()` — `MockRheology`: Setup zeros state, `N` `ApplySteps` advance it deterministically, and a write→read of `rheo_state` (checkpoint round-trip helper) reproduces it exactly. - `Test_Parse_Rheology()` — `ParseSpatialFrictionConfigString` on a `[rheology]` `kind="linear_elastic"` TOML yields `RheologyBlock::Kind::LinearElastic`; a bad kind aborts.

Acceptance criteria. - `[ ] kind="linear_elastic"` (default) $\Rightarrow$ every existing problem (TPV205/102/104/31/6 + TPV26) byte-identical to pre-change: make test green **and** a TPV205 spatial regression trace unchanged. - `[ ] test_bulk_rheology_framework` passes (no-op, factory, state round-trip, parse). - `[ ]` Checkpoint/restart of a `NumStateVars()`>0 mock reproduces `rheo_state`.

Dependencies. Depends on: Phase 0 (config skeleton). Required by: Phase 5b.

Phase 5b – DruckerPragerViscoplastic model (TPV27)

In one sentence: TPV27 gets its off-fault non-associative Drucker–Prager viscoplasticity as the first nonlinear `BulkRheology` member — an operator-split per-node return map — while TPV26 (and everything else) stays byte-identical.

Goal. Implement the spec Part-6 return map (§4.4.4) as a `BulkRheology` member, with `Setup` building the per-node effective σ^0_{eff} (§4.4.5) and `eta`(/ep) diagnostic state.

Files to create. - `constitutive/drucker_prager_core.{hpp,cpp}` — **[rev-5]** the embedding-agnostic core (§4.4.4): `dp_core::Params + dp_core::ReturnMap(s_eff[6], dt, params, s_out[6], d_eta[, d_ep[6]])  $\rightarrow$  bool` (stateless, tensor-in/out, no MFEM discretization types).

Reused verbatim by the future QD adapter. - `constitutive/drucker_prager_rheology.{hpp,cpp}` — `class DruckerPragerViscoplastic : public BulkRheology`, the **thin dynamic adapter** over the core.

Files to modify. - `spatial/code/spatial_friction.hpp/.cpp` — add `RheologyBlock::Kind::DruckerPrager + DruckerPragerParams { real_t cohesion_pa, bulk_friction, Tv_s; bool output_state; }` + `[rheology.drucker_prager]` `parse` (extend `parse_rheology`). - `constitutive/bulk_rheology_factory.cpp` — construct `DruckerPragerViscoplastic` for the `drucker_prager` kind. - `drivers/spatial_dyn_driver.cpp` — build the injected per-node `sigma0_eff` (6*ndof) by sampling the Phase-1 `Tpv2627DepthStressSource` at each bulk node, and set `ctx.sigma0_eff` before `Setup`. - TPV27 config selects `kind="drucker_prager"`.

Detailed requirements. 1. **Core** (`drucker_prager_core`): `ReturnMap` computes $m = (s_{xx} + s_{yy} + s_{zz})/3$, deviator, $J2$, $\tau = \sqrt{J2}$, $Y = \max(0, c \cdot \cos\varphi - m \cdot \sin\varphi)$, $r = -\expm1(-dt/Tv)$ ($Tv \leq 0 \Rightarrow r=1$); if $\tau > Y$, $yieldFactor = (Y/\tau - 1) \cdot r$, $s_{out} = s + yieldFactor \cdot s_{dev}$ (mean preserved), $d_eta = dt \cdot \sqrt{(0.5 \cdot \sum d\varepsilon_i^2)}$, return true; else copy $s_{out}=s$, $d_eta=0$, return false. Params precomputes $\cos\varphi$, $\sin\varphi$ from `bulk_friction`. **No state, no Q, no σ^0** — the caller supplies the effective total stress. 2. **Adapter** (`DruckerPragerViscoplastic`): `NumStateVars()==1` (`eta`; optionally 7 with `ep_ij`); `StateFieldNames()={"plastic_strain"[, "ep_xx"...]}`; `EffectiveModuli $\rightarrow$ nullptr`; `Name()=="drucker_prager"`. `Setup` caches the **injected** `ctx.sigma0_eff` (does **not** build it — §4.4.5) and zeros `eta`. 3. `ApplyStep` = per-DOF loop: $s_{eff} = sigma0_eff + Q_stress$, `dp_core::ReturnMap(s_eff, dt, params, s_new, d_eta)`, then $Q_stress[k] += (s_new[k] - s_eff[k])$ (the deviatoric change; mean preserved) and $eta[j] += d_eta$. Applied at **all** bulk

DOFs (no fault exclusion). Element-local $\Rightarrow$ no CFL change, no MPI.

Interfaces. `dp_core::ReturnMap(...); DruckerPragerViscoplastic(const DruckerPragerParams&)` (reads `ctx.sigma0_eff`); inherits `BulkRheology`.

Edge cases. $\tau==0$ (isotropic state) $\Rightarrow \tau>Y$ false ($Y\geq 0$) $\Rightarrow$ no division; $Tv\leq 0 \Rightarrow r=1$ (rate-independent); Y clamped ≥ 0 ; below-yield ($\tau\leq Y$) $\Rightarrow$ exact no-op (TPV26-equivalent bit-for-bit).

Unit tests. The golden **math** is tested against the *core* (truly standalone, no Q/mesh/spatial code — the cleanest surface and the piece the future QD adapter shares); a thinner test covers the *adapter* wiring.

- **tests/unit/test_drucker_prager_core.cpp** (Style A; links **only** `drucker_prager_core.o` + `$(MFEM_LIBS)`; fully standalone — call `dp_core::ReturnMap` on hand-built stress tensors):
 - `Test_RelaxTime()` — $r == -\exp(1(-dt/Tv))$ for several dt/Tv (incl. $dt\ll Tv$ and $dt\gg Tv \rightarrow r\rightarrow 1$); $Tv\leq 0 \Rightarrow r=1$.
 - `Test_Hypocenter_NoYield()` — effective σ^0_{eff} at 10 km depth: $m\approx -163.65$ MPa, $\tau\approx 30.15$ MPa, $Y\approx 32.40$ MPa, $\tau/Y\approx 0.930 < 1 \Rightarrow$ `ReturnMap` returns false, $s_{out}=s_{in}$ bit-identical, $d_{eta}=0$ (the spec’s “93 % of yield”).
 - `Test_Forced_Yield()` — scale the deviator so $\tau = 2\cdot Y$: `ReturnMap` returns true; mean preserved (TEST_NEAR), $s_{out} = s + (Y/\tau-1)\cdot r\cdot s_{dev}$, $d_{eta} = dt\cdot \sqrt{(0.5\cdot \sum d\varepsilon\cdot^2)} > 0$.
 - `Test_Spec_Fortran_Sample()` — reproduce the spec p.16-17 worked sample (decay= $\exp(-dt/Tv)$, $yldfac=decay+(1-decay)\cdot\tau_{lim}/\tau$) within tolerance — the byte-level acceptance oracle.
 - `Test_Regime_Agnostic()` — the same yielding tensor at coseismic $dt\sim Tv$ (partial relax, $r<1$) vs interseismic $dt\gg Tv$ ($r\approx 1$, full return to yield): confirms the kernel is valid in both dt regimes (the §4.4.8 QD-reuse property).
- **tests/unit/test_drucker_prager_rheology.cpp** (Style A; links `drucker_prager_rheology.o` + `drucker_prager_core.o` + `bulk_rheology_factory.o` + spatial set; standalone — 1-DOF Q/state):
 - `Test_Adapter_WritesBack()` — with an injected `ctx.sigma0_eff` and a yielding Q, `ApplyStep` applies $Q_{stress} += (s_{new} - s_{eff})$ (mean preserved) and increments `eta`; a below-yield Q is a **bit-exact no-op**.
 - `Test_DP_Equals_TPV26_When_Subyield()` — DP params chosen so $\tau<Y$ everywhere $\Rightarrow$ identical to `linear_elastic` (confirms gating + no spurious yielding).

Acceptance criteria. - [] `test_drucker_prager_core` passes (relax-time, no-yield-at-hypocenter, forced-yield mean-preserved + `yieldFactor` form, spec-Fortran oracle, regime-agnostic). - [] `test_drucker_prager_rheology` passes (adapter write-back + injected σ^0 + sub-yield bit-exact no-op). - [] TPV27 with sub-yield DP params is bit-identical to TPV26 (make test + regression trace). - [] Plastic strain localizes off-fault in a smoke run; no NaN/blowup; CFL unchanged. - [] **Frontera (user-approved):** TPV27 100 m/50 m vs SCEC TPV27 reference (rupture times, on-fault slip/stress, off-fault disp/vel; plasticity should reduce peak slip-rate and limit rupture vs TPV26).

Dependencies. Depends on: Phases 1, 4 (σ^0_{eff} depth formulas + outputs), 5a (framework). Required by: Phase 5c (CDBM reuses the same seat).

Phase 5c (FUTURE, not this deliverable) – continuum damage-breakage model

In one sentence: a future CDBM model drops into the same BulkRheology seat plus the two net-new pieces (§4.4.7) — no work now; the framework leaves the seat.

Reserved: a new constitutive/continuum_damage_breakage_rheology.{hpp,cpp} implementing BulkRheology, plus the EffectiveModuli() $\rightarrow$ wave-operator heterogeneous-flux feedback and the nonlocal- ξ MPI pre-pass (§4.4.7). Tracked by the companion CDBM port plan; not implemented or tested here.

1.7 6. Testing & verification strategy

This section is the **cross-cutting** view (regression contract, smokes, production). The **per-phase unit tests** — with exact test files, harness (§5.0), link sets, and golden values — live in each phase’s *Unit tests* block in §5. The lists below summarize the golden values; §5 is the authoritative test spec.

6.1 Unit tests (local; fast) – golden values from §1.2/§1.5

- **Stress source** — the 175.64/27.66/0.1575 and Ω -taper checks; frame & sign.
- **DP return map** (DruckerPragerViscoplastic, verified against the **SCEC spec** Part-6 / Fortran p.16-17; SeisSol Plasticity.cpp as a cross-check, not a parity target): (a) ComputeRelaxTime matches — $\text{expm1}(-dt/T_v)$ incl. $T_v \leq 0 \rightarrow 1$; (b) at hypocenter depth with $Q=0$ and σ^0_{eff} : $m=-163.65$ MPa, $\tau=30.15$ MPa, $\tau_{\text{ulim}}=32.40$ MPa $\Rightarrow \tau < \tau_{\text{ulim}} \Rightarrow$ **no yield, Q unchanged** (the spec’s “93 % of yield”); (c) a forced-yield case (Q pushing $\tau > \tau_{\text{ulim}}$): mean preserved, $Q+=\text{yieldFactor}\cdot s$, $\text{yieldFactor}=(\tau_{\text{ulim}}/\tau-1)\cdot r$, $\text{eta increment}=dt\cdot\sqrt{(0.5\sum d\varepsilon^2)}$; (d) vs the spec Fortran sample (p.16-17); (e) below-yield is a bit-exact no-op.
- **ResolveForcedRupture** — $T(0)=0$, monotonic, $1e9$ at $r \geq r_{\text{crit}}$, matches #if 0 values.
- **Cohesion taper** — 4.0/0.40/0.40 MPa.
- **Forced-rupture iterator parity** — TPV205 byte-exact.

6.2 Regression / byte-exactness

- TPV205 native + spatial traces unchanged (the round-6 iterator change is gated).
- `kind="linear_elastic"` (the default, incl. TPV26 and every existing problem): bit-identical to pre-change — the rheology ApplyStep is skipped (`NumStateVars()==0` gate); TPV27 with DP params forced sub-yield is also bit-identical to TPV26.
- Existing make test suite green (per CLAUDE.md baseline).

6.3 Integration smokes (local, coarse mesh, short t; $np \leq 10$ per memory)

- TPV26 end-to-end: rupture nucleates at hypocenter, propagates, surface-breaks; outputs well-formed.
- TPV27 end-to-end: same + off-fault yielding active; no NaN/blowup; CFL unchanged.
- *Per memory: no production-mesh local runs; tiny fixtures only.*

6.4 Production (Frontera; user-approved sbatch only)

- 100 m and 50 m, p1 (and p2 for convergence), 0→13 s, ADER (default).
 - Compare on-fault rupture-time contours, slip/slip-rate/stress at the 12 stations, off-fault disp/vel at the 6 body stations against published SCEC TPV26/27 reference solutions.
 - Follow memory rules: match a working sbatch's full module load + LD_LIBRARY_PATH; notify and wait for explicit approval before any Frontera submission.
-

1.8 7. Risks & open decisions

1. **Stress sign/frame remap** (highest risk) — the spec↔code axis remap and the tension↔compression negation are load-bearing. *Mitigation*: the §2.4 master table + the Phase-1 golden tests (and a one-trace comparison of on-fault n -stress/ τ sign vs TPV205) must pass before any production run.
 2. **Round-6 interior-face forced rupture** — touches a byte-exact file. *Mitigation*: gate on WaveOpLaw(), keep the TPV205 call path literally unchanged, add the parity test; verify interior vs seam $\mu(\delta, t)$ consistency in MPI ($np \geq 2$).
 3. **Surface-breaking node Pf=0** — ComputeParams gives Pf=0 at $z=0$; spec wants a half/third-element Pf. *Mitigation*: rely on C0=4 MPa near surface; if the top node misbehaves add a half-element Pf offset / min_sigma_n (verification item, not blocking).
 4. **$p \geq 2$ nodal-GLL plasticity (modal↔nodal nuance)** — SeisSol round-trips modal→nodal→modal via a Vandermonde pair (Plasticity.cpp:76–81, 212–216) evaluating the return map at a fixed nodal set; with an *invertible* (collocation) Vandermonde this is exactly a pointwise nodal update — identical to our direct nodal-GLL collocation. At $p \geq 2$ the corrected nodal values are kept as DOFs (interpolation), whereas a denser nodal set + L2 re-projection would differ slightly and can curb aliased over/under-yielding. *Mitigation*: validate p1 first (exact equivalence); for p2+ compare against the SeisSol reference and, if artifacts appear, evaluate on a denser nodal set and project back (the SeisSol pattern).
 5. **Off-fault displacement accumulator** — restart-from-0 semantics (documented in tpv6_stations.hpp); ensure resumed runs are handled or document the limitation.
 6. **Output cadence resampling** — the SCEC server requires equal time steps with high-precision time; the explicit dt is variable. *Mitigation*: resample on-fault/off-fault series to a uniform grid (~ 0.005 – 0.008 s) with 20.12E time stamps.
 7. **DP cadence** — per-macro-step (recommended, SeisSol cadence) vs per-substep. *Mitigation*: start per-macro-step; revisit only if Frontera comparison shows excess dissipation.
 8. **ConstitutiveModel subclassing** — optional, for unit-test symmetry only; the wave operator will not call its virtuals. *Decision*: standalone free function is primary; thin wrapper optional.
 9. **TPV27 mesh resolution off-fault** — plastic zone width drives the near-fault fine band. *Mitigation*: widen the size-field band for TPV27 if reference comparison shows under-resolution.
-

1.9 8. File manifest

8.1 New files

File	Purpose
spatial/code/spatial_stress.{hpp,cpp} (extend)	Tpv2627DepthStressSource
dynamic/spatial_nucleation.{hpp,cpp} (extend)	[CORR] free function ResolveForcedRupture + ForcedRupturePerD0FParams (alongside the 3 existing resolvers — NOT a SpatialFrictionResolver method)
constitutive/bulk_rheology.hpp	BulkRheology interface (the nonlinear-constitutive path)
constitutive/linear_elastic_rheology.hpp	no-op default member (keeps existing problems byte-identical)
constitutive/drucker_prager_core.{hpp,cpp}	[rev-5] embedding-agnostic DP core (yield + return map + state); reused by the future QD adapter (§4.4.8)
constitutive/drucker_prager_rheology.{hpp,cpp}	DP viscoplastic BulkRheology — thin dynamic adapter over the core (TPV27)
constitutive/bulk_rheology_factory.{hpp,cpp}	MakeBulkRheology(cfg.rheology)
dynamic/tpv2627_stations.hpp	on-fault 12-station, 8-col SCEC writer (clone tpv205_stations.hpp)
dynamic/tpv2627_surface_stations.hpp	off-fault 6-station, 7-col writer (clone TPV205SurfaceStationWriter @ tpv205_setup.hpp:213; $\int v \, dt$ idiom from tpv6_stations.hpp:151–186)
tpv26/mesh/tpv26_{200,100,50}.m.geo	surface-breaking vertical 40×20 km fault (Gmsh v2.2)
tpv26/configs/*.toml, tpv27/configs/*.toml	problem configs
tests/unit/test_tpv2627_stress_source.cpp	Phase 1: stress golden tests (175.64/27.66/0.1575, Ω taper, sign)
tests/unit/test_forced_rupture_resolver.cpp	Phase 2: T(r) tests ([CORR] adapt #if 0 @ test_spatial_friction_resolver.cpp:904–1203)
tests/unit/test_forced_rupture_iterator_parallel.cpp	Phase 2: TPV205 byte-exact + interior forced-rupture front
tests/unit/test_tpv2627_cohesion_taper.cpp	Phase 3: $C_0(z) = 4.0/2.2/0.40/0.40$ MPa
tests/unit/test_tpv2627_station_format.cpp	Phase 4: on-fault 8-col layout, header tokens, n-stress sign
tests/unit/test_rupture_time_tracker.cpp	Phase 4: first $V_{abs} > 0.001$ crossing → cplot
tests/unit/test_displacement_accumulator.cpp	Phase 4: trapezoidal $\int v \, dt$ exactness
tests/unit/test_bulk_rheology_framework.cpp	Phase 5a: interface/factory + linear-elastic no-op + state round-trip
tests/unit/test_drucker_prager_core.cpp	Phase 5b: DP core golden math (standalone; relax-time, no-yield, forced-yield, spec-Fortran, regime-agnostic)

tests/unit/test_drucker_prager_rheology.cpp Phase 5b: DP adapter wiring (Q write-back, injected σ^0 , sub-yield no-op)

8.2 Modified files

File	Change
spatial/code/spatial_friction.hpp	StressSourceKind::Tpv2627Depth (+Tpv2627DepthStressSpec); NucleationKind::ForcedRupture (+ForcedRuptureSpec); RheologyBlock + std::optional<RheologyBlock> rheology. [CORR] ResolveForcedRupture decl is NOT here (see spatial_nucleation.hpp)
spatial/code/spatial_friction.cpp	parsers (parse_stress_kind, [nucleation] kind="forced_rupture", parse_rheology/[rheology]) + validation
dynamic/friction_substep_iterator.{hpp,cpp}	round-6 fix: add real_t t_sub_end param to StepOneQP_ (.hpp:352, forward from .cpp:321) + a bool forced_rupture_member; gated LSWFrictionCoefficient_ForcedRupture on interior faces (plain-LSW branch textually unchanged)
dynamic/friction_iterator_factory.cpp	pass forced_rupture flag to the LinearSlipWeakeningIterator ctor (:28–32)
dynamic/wave_operator.hpp	[CORR] add FaultFrictionLaw::LSW_ForcedRupture (accessors GetScalarNDof/GetFESpace/GetMesh already exist; no .inl/elastic-core change)
drivers/spatial_dyn_driver.cpp	stress-source arm (:1683–1806); SetFaultFrictionLaw→LSW_ForcedRupture (:1397); real T_forced/t0 replace dummies (:1974–2002); bulk node_xyz build + injected sigma0_eff (sample the Phase-1 Tpv2627DepthStressSource at bulk nodes → ctx.sigma0_eff) + MakeBulkRheology + Setup + gated ApplyStep before t += dt_step :3365 (after Q.Swap :3364); rheo_state in Read/WriteTpv104Checkpoint (:2940/:3648/:3692); ParaView state fields (:2730–2765/:3157–3175); tpv26/tpv27 station + cplot dispatch (:3269–3300, per-step :3565)

File	Change
Makefile	new .cpp objects (spatial_nucleation already built; drucker_prager_core.cpp, drucker_prager_rheology.cpp, bulk_rheology_factory.cpp new) + 10 test targets (4-part idiom, §5.0)

8.3 Untouched byte-exact contracts (do NOT regress)

dynamic/tpv205_friction.hpp (reused, not edited), native tpv205/tpv102/tpv104 drivers, BP5/BP1/BP2 and friction/dieterich_ruina.hpp (per memory: BP5 sources are no-touch; dynamic/ is editable for TPV-style work). The TPV205 spatial regression must remain byte-identical.

1.10 Appendix A – Provenance

Built from a 12-agent code-exploration workflow (spec extraction $\times 2$ + adversarial verify; subsystem explorers for driver flow, problem template, stress system, wave operator, LSW friction, mesh, config/build; plasticity integration deep-dive + SeisSol Plasticity.cpp reference) plus direct verification of the keystone facts (Q tension-positive sign, nodal GLL basis, integrator choke-point, LSWFrictionCoefficient_ForcedRupture signature) and independent re-derivation of the 0.1575 stress ratio and 0.930 yield ratio.

SeisSol off-fault-plasticity reference (read for rev 2, v1.3.1-1760-g49bdd63e4): - src/Kernels/Plasticity.cpp — computePlasticity: modal $\rightarrow$ nodal + add initialLoading (:70-81), mean (:84-88), deviator (:93-97), $I2/\tau$ (:99-110), $\tau_{lim} = \max(0, c \cdot \cos\varphi - m \cdot \sin\varphi)$ (:114-118), yieldFactor = $(\tau_{lim}/\tau - 1) \cdot r$ (:122-132), factor = $\mu factor / (tV \cdot r)$ (:135), plastic-strain + stress update (:160-216). - src/Kernels/Plasticity.h:26-28 — computeRelaxTime(tV, dt) = $-\expm1(-dt/tV), tV \leq 0 \rightarrow 1$. - src/Model/Plasticity.h:25-82 — PlasticityData: initialLoading (effective stress, 6 comps), cohesionTimesCosAngularFriction, sinAngularFriction, $\mu factor = 1/(2 \cdot \mu^-)$, 7 output quantities. - src/Model/CommonDatastructures.h:70-80 — struct Plasticity {bulkFriction, plastCo, sXX..sXZ}. - src/Solver/TimeStepping/TimeCluster.cpp:911-985 — once-per-step, post-neighbor-integral cadence with the full timeStepSize(); applied to every plasticityEnabled cell (DR-adjacent included). - src/Initializer/ParameterDB.cpp:374 — easy binding s_xx... for the initial loading (effective stress).